

UNIVERSITÀ TELEMATICA INTERNAZIONALE UNINETTUNO

FACOLTÀ DI INGEGNERIA

Corso di Laurea Magistrale in
Ingegneria Informatica

ELABORATO FINALE

in

Progettazione del software

Sviluppo di una Pipeline di Big Data Analytics per la Classificazione in Tempo Reale di Aritmie Cardiache

CORRELATORE
Prof. Mazzei Mauro

CANDIDATO
Donadello Andrea

TODO Dedicar

Sommario

Si presenta un sistema per la classificazione *near-real-time* di flussi ECG (elettrocardiogramma) basato su un approccio **ibrido batch/streaming**.

Il problema è affrontato come una pipeline di decisione distribuita nella quale l'addestramento del modello avviene in modo *off-line* su dati storici, mentre l'inferenza è eseguita su eventi in arrivo con vincoli di latenza compatibili con scenari di monitoraggio continuo. L'architettura è stata implementata in **Scala** e combina un layer di addestramento batch, realizzato con **Apache Spark MLlib**, e un layer di inferenza in streaming, realizzato con **Spark Structured Streaming** che consuma messaggi da un broker **Kafka**.

Le *feature* utilizzate sono estratte a livello di battito dal dataset Kaggle *ECG Arrhythmia Classification Dataset* e riflettono contesto ritmico locale (intervalli **pre-RR** e **post-RR**), ampiezze delle onde P/T e del complesso QRS, durate degli intervalli PQ/QT/ST e descrittori morfologici del QRS su due derivazioni (lead II e V5). Questa rappresentazione riduce la dimensionalità del problema e consente di applicare modelli di *classical machine learning* su vettori strutturati, costruiti tramite **VectorAssembler**.

Il modello scelto per l'implementazione sperimentale è un **RandomForestClassifier** configurato con 100 alberi e profondità massima pari a 15, in modo da bilanciare capacità discriminativa, robustezza rispetto al rumore e semplicità di deployment.

La pipeline complessiva prevede quattro fasi principali:

1. Preparazione e pulizia dei dati storici per l'addestramento, comprensiva di normalizzazione delle feature e gestione degli outlier;
2. Serializzazione e deploy del modello su storage locale, così da rendere l'inferenza indipendente dalla fase di training;
3. Simulazione di ingestion in *real-time* tramite il componente **EcgPatientSimulator** che pubblica payload JSON sul topic **ecg-input-stream**;
4. Lettura dai topic Kafka, deserializzazione dinamica con **from_json**, applicazione del modello nel componente **EcgStreamingPredictor** e persistenza dei risultati in **Delta Lake** con *checkpointing* e **foreachBatch** per garantire tolleranza ai fault, recupero in caso di failure e comportamento *exactly-once* a livello di micro-batch.

La valutazione sperimentale adotta il `MulticlassClassificationEvaluator` per misurare l'accuratezza e analizzare il bilanciamento tra falsi positivi e falsi negativi, con particolare attenzione all'impatto clinico di quest'ultimi.

Accanto alla qualità predittiva, si discutono *latenze end-to-end*, *throughput* e comportamento della pipeline in presenza di componenti disaccoppiati, evidenziando il ruolo della messaggistica asincrona e dello storage transazionale.

I capitoli sui risultati e sulle conclusioni discutono metriche, latenza e limiti del prototipo; tra le estensioni possibili figurano modelli deep sul segnale raw, feature su edge device e dashboard su Delta Lake.

Indice

1	Introduzione	12
1.1	Contesto e motivazione	12
1.2	Approccio e scelte tecnologiche	12
1.3	Obiettivi	13
1.4	Contributi	13
1.5	Struttura della tesi	14
2	Dominio Applicativo e Dataset	15
2.1	Dominio applicativo: il segnale ECG	15
2.1.1	Elettrofisiologia e morfologia del battito	15
2.1.2	Variabilità della frequenza cardiaca	16
2.1.3	Rappresentazione computazionale e feature	17
2.1.4	Preprocessing e qualità del segnale	17
2.2	Dataset sperimentale: MIT-BIH tabellare (Kaggle)	18
2.2.1	Origine e caratteristiche del corpus PhysioNet	18
2.2.2	Schema del file CSV e volume dati	19
2.2.3	Schema di raggruppamento AAMI e etichette nel CSV	19
2.2.4	Sbilanciamento tra classi	20
2.2.5	Tipologie di anomalie contenute e limiti	20
2.3	Edge Computing e ruolo del simulatore	21

2.3.1	Funzione metodologica del simulatore	21
2.3.2	Scenario evolutivo: digital twin	22
2.4	Considerazioni etiche e implicazioni cliniche	22
3	Stack Tecnologico	23
3.1	Il linguaggio Scala: programmazione funzionale applicata ai Big Data	23
3.1.1	Immutabilità, composizione e type system	23
3.1.2	Confronto con alternative	24
3.2	Apache Spark: Core, Spark SQL e MLlib	24
3.2.1	Modello di esecuzione	24
3.2.2	Spark SQL, Catalyst e Tungsten	24
3.2.3	MLlib, Pipeline API e confronto con alternative	25
3.2.4	Structured Streaming: semantica temporale e trigger	25
3.3	Apache Kafka: architettura publish-subscribe e disaccoppiamento	26
3.3.1	Architettura log-centrica	26
3.3.2	Partizioni, consumer group e parallelismo	26
3.3.3	Modalità KRaft e deprecazione di ZooKeeper	27
3.3.4	Garanzie di consegna ed exactly-once	27
3.4	Delta Lake: il paradigma Data Lakehouse	27
3.4.1	Limiti di Parquet puro	27
3.4.2	Transaction log, ACID e schema	28
3.4.3	Time travel, VACUUM e audit clinico	28
3.4.4	Integrazione con Spark	28
3.5	Sintesi: requisiti e tecnologie adottate	28
4	Caso di studio	30
4.1	Pattern architetturali: Lambda, Kappa e posizionamento del progetto	30

4.2	Panoramica della pipeline	31
4.2.1	Flusso granulare dei dati	32
4.3	Vista fisica e deployment	32
4.3.1	Limiti del setup mono-nodo	33
4.4	Fase di addestramento: EcgModelTrainer	33
4.4.1	Flusso operativo	33
4.4.2	Scelta del classificatore e alternative	33
4.4.3	Output del trainer	34
4.5	Simulazione IoT: EcgPatientSimulator	34
4.5.1	Logica di produzione	34
4.5.2	Schema del payload e impatto sulla banda	35
4.5.3	Estensioni e limiti	35
4.6	Inferenza in tempo reale: EcgStreamingPredictor	35
4.6.1	Bootstrap dello schema JSON	35
4.6.2	Schema Registry e contratto dati	35
4.6.3	Pipeline di inferenza	36
4.6.4	Ciclo di vita del modello in produzione	36
4.6.5	Fault tolerance e exactly-once	36
4.7	Orchestrazione: AppRunner	36
4.8	Requisiti non funzionali	37
4.8.1	Disaccoppiamento	37
4.8.2	Idempotenza e consistenza	37
4.8.3	Scalabilità	37
4.8.4	Evolvibilità dello schema	37
4.8.5	Observability	38
4.8.6	Sicurezza	38

5	Implementazione e Dettagli Tecnici	39
5.1	Organizzazione del codice sorgente	39
5.1.1	Centralizzazione della configurazione	39
5.2	Build e gestione delle dipendenze	40
5.3	Infrastruttura Docker e broker Kafka	40
5.4	Configurazione <code>SparkSession</code>	41
5.5	Data pipeline e machine learning	41
5.5.1	Preprocessing e feature engineering	41
5.5.2	Split, addestramento e valutazione	42
5.5.3	Iperparametri Random Forest	42
5.5.4	Serializzazione e persistenza del modello	43
5.6	Gestione dello streaming	43
5.6.1	Parsing JSON e schema dinamico	43
5.6.2	Trasformazione e proiezione output	44
5.6.3	Checkpointing e fault tolerance	44
5.6.4	<code>foreachBatch</code> rispetto al sink Delta nativo	45
5.7	Storage e sink: Delta Lake	45
5.7.1	Configurazione sink	45
5.7.2	Operazioni successive	45
5.8	Producer Kafka del simulatore	46
5.8.1	Perché <code>KafkaProducer</code> puro	46
5.8.2	Limiti dell'implementazione	46
5.9	Orchestrazione e ciclo di vita	47
5.9.1	Banner di avvio	47
5.10	Gestione errori e resilienza	47
5.11	Strategie di testing	48

5.12	Riepilogo implementativo	48
6	Analisi dei Risultati e Test	49
6.1	Protocollo sperimentale	49
6.1.1	Ambiente hardware e software	49
6.1.2	Dataset e parametri di training	49
6.1.3	Procedura di esecuzione	50
6.2	Performance del modello	50
6.2.1	Accuratezza globale	50
6.2.2	Metriche per super-classe AAMI	50
6.2.3	Matrice di confusione	51
6.2.4	Considerazioni sull'accuratezza	51
6.3	Analisi degli errori e impatto clinico	51
6.3.1	Scenari operativi e sensibilità richiesta	51
6.3.2	Allineamento con AAMI EC57 e sbilanciamento del dataset	52
6.3.3	Falsi negativi sulle ectopie ventricolari	52
6.3.4	Falsi positivi e carico operativo	52
6.4	Analisi della latenza	53
6.4.1	Definizione end-to-end	53
6.4.2	Decomposizione (valori provvisori)	53
6.4.3	Warmup iniziale	53
6.5	Throughput e back-pressure	53
6.5.1	Misura in condizioni nominali	53
6.5.2	Effetto delle partizioni Kafka	54
6.5.3	Back-pressure	54
6.6	Scalabilità	54
6.6.1	Limiti del benchmark mono-nodo	54

6.6.2	Proiezione su cluster	54
6.7	Riproducibilità	54
6.8	Sintesi dei risultati sperimentali	55
7	Conclusioni	56
7.1	Sintesi dei risultati raggiunti	56
7.1.1	Obiettivo 1: pipeline end-to-end	56
7.1.2	Obiettivo 2: addestramento batch con Random Forest	56
7.1.3	Obiettivo 3: ingestion simulata	56
7.1.4	Obiettivo 4: inferenza streaming con fault tolerance	57
7.1.5	Obiettivo 5: valutazione sperimentale	57
7.1.6	Punti di forza	57
7.1.7	Limiti onesti	57
7.2	Possibili estensioni	57
7.2.1	Deep learning sul segnale raw	57
7.2.2	Edge AI e feature extraction distribuita	58
7.2.3	Dashboard real-time e analisi storica	58
7.2.4	Schema Registry e contratti dati	58
7.2.5	MLOps e retraining continuo	58
7.2.6	Validazione clinica e conformità normativa	58
7.3	Riflessione critica sulle scelte architetturali	59
7.4	Collocazione nel panorama e-Health e Big Data	59
7.5	Considerazioni conclusive	59

Elenco delle figure

2.1	Morfologia schematica di un battito ECG in ritmo sinusale: deflessioni P, Q, R, S e T; intervalli PR e QT; durata del complesso QRS; segmento ST.	16
3.1	Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.	25
4.1	Schema concettuale dell'architettura Lambda: batch e speed layer alimentano il serving.	30
4.2	Flusso logico: training batch, ingestion simulata e inferenza streaming. Linee tratteggiate: orchestrazione AppRunner	32
4.3	Sequenza di avvio orchestrata da AppRunner : training condizionale, predictor in thread demone, attesa, simulatore, join.	32
4.4	Deployment mono-nodo: tre processi JVM Spark (<code>local[*]</code>) e un container Docker per Kafka.	33
6.1	F1-score per super-classe. Valori da sostituire con misure sperimentali definitive.	51

Elenco delle tabelle

2.1	Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.	19
2.2	Raggruppamento AAMI delle annotazioni MIT-BIH (sintesi del contenuto clinico).	19
2.3	Feature per derivazione nel CSV MIT-BIH (prefissi 0_ e 1_).	20
3.1	Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.	27
3.2	Mappatura requisiti-tecnologie.	28
4.1	Confronto sintetico Lambda, Kappa e impostazione del progetto.	31
5.1	Riepilogo componenti e scelte implementative.	48
6.1	Configurazione sperimentale (placeholder).	49
6.2	Metriche per super-classe AAMI (valori provvisori).	50
6.3	Decomposizione latenza end-to-end (ms, valori provvisori).	53

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Le patologie cardiovascolari rappresentano una delle prime cause di morbidità e mortalità a livello globale; il monitoraggio continuo dell'attività cardiaca tramite segnali ECG è pertanto centrale per la diagnosi precoce e la gestione clinica.

In molti scenari applicativi, l'interpretazione del segnale non può essere affidata esclusivamente a una valutazione umana *ex post*, poiché il valore informativo del battito risiede anche nella *tempestività* con cui viene rilevata un'anomalia. Da questo punto di vista, l'analisi automatica dei battiti cardiaci pone sfide tecniche sostanziali:

- La necessità di elaborazione **near-real-time**,
- La gestione di flussi ad alta velocità,
- La robustezza rispetto a fault e perdita di pacchetti,
- La capacità di mantenere *consistenza* tra fasi di ingestione, predizione e persistenza.

Nel dominio medico i trade-off tra **precisione** e **sensibilità** ricoprono un ruolo critico. I falsi negativi possono portare a omissioni diagnostiche rilevanti, mentre i falsi positivi possono introdurre rumore operativo e aumentare il carico di verifica manuale.

Ne deriva l'esigenza di una pipeline che non si limiti a produrre una classificazione accurata, ma che sia anche *interpretabile* dal punto di vista architetturale, scalabile nel numero di sorgenti e adatta a operare su infrastrutture distribuite. In tale prospettiva, il progetto affronta il problema della classificazione di battiti ECG come un caso di studio per l'integrazione tra **machine learning classico** e **data streaming**.

1.2 Approccio e scelte tecnologiche

In questo lavoro si affronta il problema della classificazione di battiti ECG in un contesto di streaming, privilegiando un'architettura che consenta **disaccoppiamento** tra ingestione e

processing, resilienza ai guasti e scalabilità orizzontale.

La soluzione proposta impiega tecnologie consolidate nello stack Big Data:

- **Scala**: implementazione del sistema e dei componenti della pipeline,
- **Apache Kafka**: broker Pub/Sub per la messaggistica asincrona,
- **Apache Spark**: elaborazione batch (MLlib) e streaming (Structured Streaming),
- **Delta Lake**: sink transazionale per la persistenza dei risultati.

Questa combinazione consente di separare in modo netto la responsabilità di acquisizione dei dati dalla responsabilità di inferenza, evitando che il carico di una fase condizioni in maniera diretta l'altra.

La scelta di una soluzione *ibrida batch/streaming* risponde a un'esigenza precisa: i modelli predittivi possono essere addestrati su archivi storici con costi computazionali più elevati, mentre l'inferenza deve operare con latenza contenuta e comportamento deterministico. Il progetto adotta quindi una pipeline che riproduce un ciclo realistico di produzione dei dati, addestramento e serving, offrendo un contesto coerente per valutare non soltanto la qualità del classificatore, ma anche la solidità dell'infrastruttura sottostante.

1.3 Obiettivi

Gli obiettivi perseguiti sono i seguenti:

1. Progettare e implementare una **pipeline end-to-end** per la classificazione di battiti ECG in near-real-time, rispettando requisiti di tolleranza ai fault, scalabilità e disaccoppiamento tra le componenti.
2. Sviluppare un processo di **addestramento batch** per un modello basato su Random Forest, includendo fasi di feature engineering, selezione delle variabili e validazione sperimentale.
3. Implementare un meccanismo di **ingestion realistico** tramite un simulatore (**EcgPatientSimulator**) che converte righe del CSV preprocessato (derivato dal MIT-BIH) in payload JSON e li pubblica su Kafka.
4. Eseguire **inferenza in streaming** con Spark Structured Streaming, garantendo atomicità delle scritture su Delta Lake tramite checkpointing e **foreachBatch**, così da preservare la continuità operativa in presenza di interruzioni.
5. Valutare la qualità del modello con metriche di classificazione e analizzare *latenze*, *throughput* e comportamento del sistema al variare del volume dei messaggi.

1.4 Contributi

Si elencano i contributi principali del lavoro:

- **Realizzazione di una pipeline sperimentale completa** in Scala che integra training batch e inferenza in streaming su dati ECG simulati, mantenendo la separazione tra livelli di acquisizione, elaborazione e persistenza.
- **Approccio pragmatico al feature engineering** su battiti cardiaci (intervalli `pre-RR/post-RR`, descrittori morfologici del QRS, durate PQ/QT/ST e coefficienti `qrs_morph` su due derivazioni) e definizione di uno schema JSON compatto per l'ingestion in Kafka.
- Dimostrazione dell'integrazione di Spark Structured Streaming con schema dinamico (`from_json`), checkpointing e scritture transazionali su Delta Lake per garantire resilienza, consistenza e ripetibilità delle elaborazioni.
- Analisi sperimentale completa della performance del modello (tramite `MulticlassClassificationEvaluator`) e delle proprietà non funzionali della pipeline, con attenzione a latenze, throughput e stabilità operativa.
- Formalizzazione di un caso di studio in cui una stessa architettura può essere riletta sia come sistema di prototipazione accademica sia come base per un futuro deployment in ambito e-Health.

1.5 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 1** descrive il dominio applicativo e il dataset utilizzato (MIT-BIH in versione tabellare da Kaggle), includendo considerazioni su feature cliniche, preprocessing del segnale e paradigma Edge Computing.
- **Capitolo 2** giustifica le scelte tecnologiche adottate, soffermandosi sui vantaggi di Scala, Spark, Kafka KRaft e Delta Lake nel contesto di pipeline distribuite.
- **Capitolo 3** illustra l'architettura complessiva e la scomposizione in tre moduli principali: `EcgModelTrainer` (batch), `EcgPatientSimulator` (ingestion) e `EcgStreamingPredictor` (streaming).
- **Capitolo 4** approfondisce dettagli implementativi, mostrando frammenti di codice e spiegando le scelte di configurazione della ML pipeline e del producer Kafka.
- **Capitolo 5** presenta la valutazione sperimentale, confrontando metriche di accuratezza, analisi di falsi positivi/negativi, e misure di latenze e throughput; infine, le conclusioni espongono valutazioni complessive e possibili linee di estensione verso modelli deep e integrazione analitica con Delta Lake.

I sorgenti del repository (`EcgModelTrainer`, `EcgPatientSimulator`, `EcgStreamingPredictor`) sono richiamati nei capitoli di implementazione e risultati, così da legare teoria e codice effettivamente scritto.

Capitolo 2

Dominio Applicativo e Dataset

2.1 Dominio applicativo: il segnale ECG

Il segnale elettrocardiografico (ECG) rappresenta la proiezione elettrica dell'attività cardiaca misurata sulla superficie corporea. Dal punto di vista fisiologico, esso riflette la sequenza di depolarizzazione e ripolarizzazione delle camere cardiache e consente di osservare, con risoluzione temporale elevata, eventi che sono spesso immediatamente correlati allo stato elettrico del miocardio [16].

2.1.1 Elettrofisiologia e morfologia del battito

L'attivazione elettrica del cuore origina nel **nodo seno-atriale** (SA), che innesca la depolarizzazione degli atri. Il fronte di onda raggiunge poi i ventricoli attraverso il **nodo atrioventricolare** (AV), il **fascio di His** e il sistema delle **fibre di Purkinje**, che distribuiscono l'impulso sul miocardio ventricolare in modo coordinato. La sequenza temporale di queste fasi si traduce, sul tracciato, in segmenti e intervalli con significato clinico ben definito.

La morfologia tipica di un battito in ritmo sinusale è costituita dalle onde **P**, **QRS** e **T**: l'onda P corrisponde alla depolarizzazione atriale; il complesso QRS alla depolarizzazione ventricolare (con i deflessi Q, R e S che descrivono l'attivazione sequenziale del setto e delle pareti ventricolari); l'onda T alla ripolarizzazione ventricolare. Tra questi deflessi si misurano il **segmento PR** (conduzione atrioventricolare), il **segmento ST** (fase di ripolarizzazione precoce ventricolare) e l'**intervallo QT** (durata dell'attività ventricolare totale), spesso corretto per la frequenza cardiaca (QTc) nelle valutazioni cliniche.

La Figura 2.1 sintetizza, in forma schematica, la relazione tra fasi elettriche e morfologia osservabile. In applicazioni automatiche, la forma locale del QRS e la durata degli intervalli costituiscono descrittori ad alta densità informativa per la distinzione tra battiti normali e patologici [7].

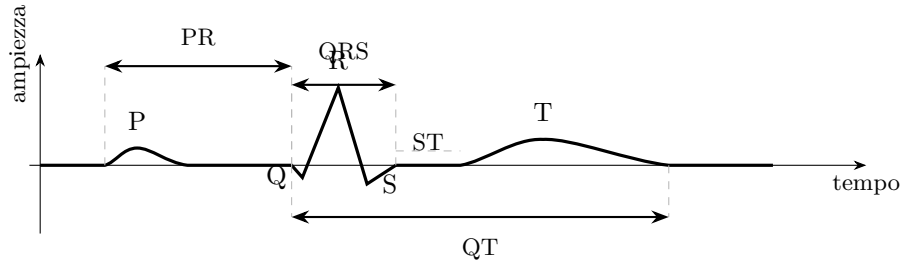


Figura 2.1: Morfologia schematica di un battito ECG in ritmo sinusale: deflessioni P, Q, R, S e T; intervalli PR e QT; durata del complesso QRS; segmento ST.

Gli *intervalli RR* (distanza temporale tra picchi R consecutivi) sono la misura principale per la frequenza cardiaca e per la *Heart Rate Variability* (HRV). In presenza di aritmie, la distribuzione degli RR perde regolarità: ectopie, pause o tachicardie producono sequenze che un classificatore può sfruttare insieme alla morfologia del battito isolato [7], [26].

2.1.2 Variabilità della frequenza cardiaca

La HRV quantifica le fluttuazioni dell'intervallo RR attorno al valore medio e riflette, in parte, l'equilibrio tra i sistemi simpatico e parasimpatico. Le linee guida del Task Force del 1996 [26] distinguono due famiglie di misure, entrambe rilevanti per il feature engineering adottato in questo lavoro.

Nel **dominio del tempo**, le metriche più diffuse includono:

- **SDNN**: deviazione standard degli intervalli NN (battiti di origine sinusale);
- **RMSSD**: radice quadrata della media dei quadrati delle differenze successive tra intervalli NN;
- **pNN50**: percentuale di coppie di intervalli NN consecutivi la cui differenza supera 50 ms.

RMSSD e pNN50 sono particolarmente sensibili alle componenti ad alta frequenza, associate all'attività vagale a breve termine.

Nel **dominio della frequenza**, lo spettro della serie RR (tipicamente ricampionata e detrendata) viene suddiviso in bande: la componente **LF** (circa 0.04–0.15 Hz) e la componente **HF** (circa 0.15–0.40 Hz), quest'ultima legata alla respirazione e al tono parasimpatico. Il **rapporto LF/HF** è usato in letteratura come indicatore sintetico dell'equilibrio autonomico, sebbene la sua interpretazione clinica richieda cautela e contesto del paziente [26].

Nel dataset impiegato, metriche HRV globali (SDNN, RMSSD, spettro LF/HF) non sono calcolate in tempo reale dalla pipeline Scala; il CSV contiene però intervalli RR locali (**pre-RR**, **post-RR**) e descrittori morfologici già estratti offline dal segnale grezzo. Il classificatore opera quindi su un livello di astrazione che unifica morfologia istantanea e contesto ritmico immediato attorno al battito corrente.

2.1.3 Rappresentazione computazionale e feature

Dal punto di vista computazionale, il battito ECG può essere rappresentato come una **sequenza multiscala**: una componente temporale (distanza tra eventi) e una componente morfologica (forma del tracciato locale). Questa doppia prospettiva consente di trasformare un segnale continuo in un insieme di osservazioni etichettate, ciascuna associata a un istante clinicamente significativo, adatto a modelli supervisionati che richiedono vettori numerici stabili nel tempo.

Nel file `MIT-BIH Arrhythmia Database.csv` del corpus Kaggle [23], [24], ogni riga descrive un singolo battito con **32 feature numeriche** (16 per ciascuna delle due derivazioni, prefissi `0_` per lead II e `1_` per lead V5), oltre a **record** (identificativo registrazione) e **type** (etichetta):

- **Contesto ritmico**: `pre-RR` e `post-RR` (intervalli R–R rispettivamente precedente e successivo al battito corrente, in ms) [7].
- **Morfologia delle onde**: ampiezze `pPeak`, `qPeak`, `rPeak`, `sPeak`, `tPeak`.
- **Durata degli intervalli**: `pq_interval`, `qrs_interval`, `st_interval`, `qt_interval`.
- **Forma del QRS**: cinque coefficienti `qrs_morph0`–`qrs_morph4` che riassumono la morfologia del complesso [8].

Tali attributi costituiscono il *vettore di ingresso* per modelli di machine learning classico (Random Forest nel caso di studio) e sono prodotti da una catena di preprocessing che precede l’addestramento batch. La scelta di non trasmettere il segnale raw riduce la complessità del modello, limita il volume dei messaggi Kafka e rende esplicita la semantica di ogni campo nel payload JSON.

2.1.4 Preprocessing e qualità del segnale

La qualità del segnale condiziona in modo diretto l’affidabilità delle feature. Prima della segmentazione beat-to-beat, la letteratura propone una sequenza di operazioni che, nel nostro flusso, sono applicate offline sulla fase di costruzione del CSV, ma che in un deployment edge replicherebbero lo stesso ordine logico [8], [22].

Filtraggio. Un **filtro passa-alto** (tipicamente con frequenza di taglio intorno a 0.5 Hz) attenua il *baseline wander*, derivante da respirazione, movimento del paziente o deriva degli elettrodi. Un **filtro notch** centrato a 50 Hz o 60 Hz rimuove l’interferenza della rete elettrica (*power-line interference*). Un **filtro passa-basso** (spesso tra 35 e 40 Hz, coerente con la banda utile del MIT–BIH) limita il rumore muscolare e le componenti ad alta frequenza poco informative per la classificazione dei battiti. In pratica, la combinazione di questi stadi equivale a un filtraggio passa-banda nell’intervallo 0.5–40 Hz citato nelle specifiche del database.

Rilevamento dei picchi R. L’algoritmo di *Pan–Tompkins* [22] è un riferimento consolidato per la detection in tempo quasi reale. La pipeline prevede: (i) filtraggio passa-banda; (ii) derivata per enfatizzare le pendenze del QRS; (iii) elevamento al quadrato per uniformare la polarità; (iv) integrazione a finestra mobile per smussare il segnale; (v) soglia adattiva su due livelli per

distinguere picchi veri da artefatti. La robustezza dipende dalla calibrazione delle soglie e dalla qualità del tracciato: in presenza di aritmie con morfologie atipiche, un detector generico può richiedere post-processing o regole cliniche aggiuntive.

Indice di qualità del segnale. Per evitare che artefatti di movimento o contatto degradino le feature, si adottano *Signal Quality Index* (SQI) che combinano metriche nel dominio del tempo, della frequenza o della forma d'onda [15]. Battiti associati a tratti di segnale con SQI insufficiente possono essere scartati, marcati come *unknown* o inviati su un canale di quarantena, riducendo la propagazione di errori verso il classificatore. In una pipeline distribuita, questa valutazione ha anche valenza architetturale: filtrare a monte significa trasmettere verso il broker solo eventi affidabili e alleggerire il carico del livello streaming.

Il preprocessing può dunque essere letto come **normalizzazione semantica**: il segnale fisiologico diventa una rappresentazione tabellare allineata al modello ML. In produzione, la fase può risiedere sul dispositivo wearable o su un gateway, così da inviare al data center feature già validate anziché campioni raw ad alta frequenza.

2.2 Dataset sperimentale: MIT–BIH tabellare (Kaggle)

Per la fase sperimentale si utilizza la versione tabellare del **MIT–BIH Arrhythmia Database** pubblicata nel corpus Kaggle *ECG Arrhythmia Classification Dataset* [23], [24], derivato dalla raccolta originale su PhysioNet [11], [19]. Il bundle Kaggle contiene più file CSV (MIT–BIH, MIT–BIH Supraventricular, INCART, SCD Holter); il prototipo Scala legge tramite `DATASET_PATH` i file `*.csv` presenti nella directory configurata: nelle sperimentazioni descritte in questa tesi si assume il file `MIT–BIH Arrhythmia Database.csv`, così da restare allineati al benchmark MIT–BIH senza mescolare registrazioni di altri database.

2.2.1 Origine e caratteristiche del corpus PhysioNet

Il MIT–BIH originale [11], [19] è una raccolta acquisita tra il 1975 e il 1979 presso il Beth Israel Hospital (oggi Beth Israel Deaconess Medical Center), con campionamento a 360 Hz e risoluzione di 11 bit su circa 10 mV di dinamica. Il corpus è diventato, nel tempo, uno *benchmark* de facto per la classificazione automatica dei battiti.

Le proprietà salienti del database grezzo sono le seguenti:

- 48 registrazioni di circa 30 minuti ciascuna, da 47 soggetti (due tracce per un paziente).
- Due canali per registrazione, derivazioni differenti (tipicamente lead I e una derivazione del lead II modificato).
- Annotazioni beat-to-beat curate da più esperti, con codici simbolici per oltre venti categorie di battiti ed eventi non ritmici.

La versione CSV impiegata nel progetto non contiene campioni raw del segnale, bensì battiti già segmentati con feature estratte offline (filtraggio, rilevamento QRS, calcolo di intervalli e

descrittori morfologici) [7], [24]: la pipeline Scala legge questi archivi senza ripetere sul cluster l'intera catena descritta in §2.1.4.

2.2.2 Schema del file CSV e volume dati

Il file MIT-BIH `Arrhythmia Database.csv` contiene **100 689** battiti da **44** registrazioni (colonna `record`), con **34** colonne totali (identificativo, etichetta e 32 feature). Quattro tracce presenti nel MIT-BIH grezzo non compaiono nel CSV preprocessato, probabilmente escluse in fase di estrazione per qualità del segnale o annotazioni incomplete. Ogni riga corrisponde a un battito; le feature duplicano la stessa semantica su due derivazioni (lead II, prefisso `0_`; lead V5, prefisso `1_`), come riassunto in Tabella 2.3.

2.2.3 Schema di raggruppamento AAMI e etichette nel CSV

Per confrontare i risultati con la letteratura clinica e con lo standard AAMI EC57:2012 [5], le annotazioni originali del MIT-BIH vengono raggruppate in cinque **super-classi**, come proposto da de Chazal et al. [7] e adottato nel Capitolo 6. Nel CSV Kaggle, la colonna `type` adotta una nomenclatura leggermente diversa dalle sigle AAMI letterali, ma con corrispondenza biunivoca (Tabella 2.1).

Tabella 2.1: Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.

CSV (<code>type</code>)	AAMI	Significato
N	N	Normale
SVEB	S	Ectopia supraventricolare
VEB	V	Ectopia ventricolare
F	F	Fusione
Q	Q	Non classificabile

Tabella 2.2: Raggruppamento AAMI delle annotazioni MIT-BIH (sintesi del contenuto clinico).

Classe	Sigla	Contenuto tipico
N	Normale	Battiti sinusali, blocchi di branca sinistro (L), blocchi di branca destro (R)
S	Supraventricolare	Ectopie atriali (A), aberranti (a), nodali (J), ritmi supraventricolari
V	Ventricolare	Contrazioni premature ventricolari (V), ectopie ventricolari (E)
F	Fusion	Battiti di fusione (F)
Q	Unknown	Non classificabili, rumore marcato (/), altre etichette non mappate

La Tabella 2.2 non esaurisce ogni codice del database, ma fissa il vocabolario con cui si discutono metriche e errori clinici nel resto della tesi. Lo `StringIndexer` di Spark ML indicizza i valori testuali effettivi (N, SVEB, VEB, F, Q) senza rinomina esplicita verso le sigle AAMI: la mappatura resta a carico dell’analisi dei risultati.

Tabella 2.3: Feature per derivazione nel CSV MIT-BIH (prefissi 0_ e 1_).

Suffisso colonna	Contenuto
pre-RR, post-RR	Intervalli R-R precedente e successivo (ms)
pPeak ... tPeak	Ampiezze onde P, Q, R, S, T
pq_interval, ..., qt_interval	Durate PQ, QRS, ST, QT (ms)
qrs_morph0-qrs_morph4	Descrittori morfologici del complesso QRS

2.2.4 Sbilanciamento tra classi

La distribuzione dei battiti nel file MIT-BIH `Arrhythmia Database.csv` è fortemente *sbilanciata* (conteggi assoluti sul corpus completo): N 90 083 (89,5%), VEB 7 009 (7,0%), SVEB 2 779 (2,8%), F 803 (0,8%), Q 15 (<0,1%). Tale proporzione riflette la prevalenza clinica dei ritmi normali in registrazioni ambulatoriali selezionate, ma penalizza i modelli che ottimizzano l’accuratezza globale senza pesi per classe.

Durante l’addestramento si possono considerare **class weighting**, undersampling della classe maggioritaria o oversampling sintetico delle minoritarie; ciascuna strategia introduce trade-off (perdita di informazione, rischio di overfitting sulle classi rare, tempi di training più lunghi). Nel prototipo attuale non è implementato alcun ribilanciamento esplicito: la scelta è coerente con l’obiettivo di dimostrare l’architettura end-to-end, ma va tenuta presente nell’interpretazione delle metriche del Capitolo 6.

Dal punto di vista sperimentale, il MIT-BIH offre un compromesso favorevole tra complessità e riproducibilità: registrazioni sufficientemente eterogenee per stressare la pipeline, ma di dimensione gestibile in un contesto accademico in cui l’obiettivo principale non è il record di performance isolato, bensì la validazione di un flusso integrato dal dato storico allo streaming.

2.2.5 Tipologie di anomalie contenute e limiti

Le annotazioni includono ectopie ventricolari (PVC), alterazioni di conduzione, battiti supraventricolari e, in alcune tracce, eventi rari o tratti rumorosi che mettono alla prova la segmentazione. Il database non copre tutte le condizioni cliniche: aritmie prolungate come la fibrillazione atriale persistente o pattern legati a insufficienza cardiaca avanzata possono essere sottorappresentati.

La *generalizzabilità* verso altre popolazioni (età, sesso, patologia, dispositivo di acquisizione) non va assunta per default. I pazienti del MIT-BIH sono in gran parte adulti ricoverati o in follow-up cardiologico negli Stati Uniti della fine degli anni Settanta; un modello addestrato solo su questa distribuzione può degradare su cohort europee, pediatriche o su segnali da wearable consumer con banda e rumore differenti.

Per un classificatore supervisionato, la coerenza tra annotazioni, segmentazione e feature è tanto importante quanto l'algoritmo di apprendimento: errori a monte si propagano come etichette rumorose o descrittori distorti, limitando il soffitto di performance raggiungibile anche con modelli più complessi.

2.3 Edge Computing e ruolo del simulatore

Nel monitoraggio cardiaco continuo, la posizione dell'elaborazione lungo il percorso dato-cloud non è banale. Si distinguono almeno tre livelli [12]:

- **Cloud:** capacità computazionale elevata, storage centralizzato, latenza di rete non trascurabile;
- **Fog:** nodi intermedi (ospedale, edge data center) che aggregano flussi da più dispositivi;
- **Edge:** elaborazione sul wearable, sul Holter o sul gateway domestico, a ridosso del sensore.

Per l'ECG, spostare filtraggio, QRS detection e estrazione delle feature verso l'edge risponde a tre esigenze concrete: ridurre la latenza tra evento cardiaco e allarme; diminuire il traffico (un battito con 32 feature numeriche occupa ordini di grandezza in meno di un secondo di segnale a 360 Hz); limitare l'esposizione di dati biometrici raw oltre il perimetro controllato del paziente.

Dispositivi come Holter multicanale, patch ECG monouso e braccialetti con singola derivazione implementano già, in parte, questa logica: acquisiscono a alta frequenza localmente, poi trasmettono riassunti, eventi o tratti selezionati. Il progetto di tesi non dispone di hardware dedicato; il componente `EcgPatientSimulator` emula il comportamento di un producer edge che pubblica feature già calcolate sul topic Kafka `ecg-input-stream`, in formato JSON.

2.3.1 Funzione metodologica del simulatore

Il simulatore consente di esercitare la pipeline end-to-end in laboratorio: stesso schema colonne del CSV di training, stesso topic, stesso consumer Spark, senza dipendenze da firmware o protocolli proprietari. In uno scenario operativo, lo sostituirebbe un modulo on-device (o un gateway) con `KafkaProducer` o equivalente MQTT verso il broker.

L'approccio edge comporta vincoli noti: CPU e memoria limitate, consumo energetico, necessità di aggiornare modelli e soglie senza interrompere l'acquisizione. Per la tesi, questi aspetti restano fuori dall'implementazione, ma orientano le scelte architetturali del Capitolo 4 (payload compatto, disaccoppiamento tramite Kafka, training batch separato dall'inferenza streaming).

La riproducibilità degli esperimenti ne beneficia: la sequenza di messaggi può essere rigenerata con la stessa logica (`limit(1000)`, intervallo di un secondo), permettendo confronti tra configurazioni del predictor o del checkpoint senza variabilità legata a dispositivi reali.

2.3.2 Scenario evolutivo: digital twin

A orizzonte più ampio, i flussi ECG da molteplici fonti potrebbero alimentare un *digital twin* cardiaco del paziente: modello che integra storico clinico, eventi in tempo quasi reale e predizioni per simulare risposte a terapie o per prioritizzare alert. Il prototipo descritto in questa tesi costituisce un mattone elementare di tale visione—ingestion, classificazione, persistenza versionata—senza pretendere di realizzarne la complessità clinica e regolatoria.

2.4 Considerazioni etiche e implicazioni cliniche

Nell'applicazione di modelli ML a dati clinici, l'impatto di **falsi negativi** e **falsi positivi** va valutato prima delle metriche aggregate. Un falso negativo su una ectopia ventricolare ripetuta può ritardare un intervento; un falso positivo aumenta il carico di revisione per il personale e, nel lungo periodo, riduce la fiducia nel sistema di alert.

Il problema è dunque operativo oltre che statistico: output, latenza e priorità degli allarmi devono essere allineati al contesto (telemetria domiciliare, reparto, terapia intensiva), dove i trade-off tra sensibilità e specificità non coincidono.

Quadro regolatorio. In Europa, software destinato a finalità mediche può rientrare nel campo del Regolamento (UE) 2017/745 sui dispositivi medici (MDR) [10]. I sistemi di supporto alla diagnosi basati su algoritmi—spesso indicati come *Software as a Medical Device* (SaMD)—richiedono gestione del rischio, tracciabilità, validazione clinica e monitoraggio post-market. Il prototipo di questa tesi non è certificato né destinato all'uso clinico; le considerazioni regolatorie servono a inquadrare il divario tra dimostrazione accademica e prodotto deployabile.

Validazione, interpretabilità e bias. Un'adozione responsabile presuppone validazione su popolazioni e dispositivi diversi da quelli del training, documentazione delle prestazioni per sottogruppo e strumenti di interpretabilità (importanza delle feature, analisi degli errori per classe AAMI). Il MIT-BIH, per quanto prezioso, introduce *bias* demografico e tecnologico: campione storico, non necessariamente rappresentativo dell'utenza attuale né delle acquisizioni da wearable.

Infine, la pipeline deve essere leggibile da chi risponde dei dati: finalità del trattamento, minimizzazione (feature anziché raw dove possibile), retention su Delta Lake e accessi al broker vanno progettati con pari attenzione rispetto all'accuratezza del classificatore. Senza questi accorgimenti, anche un modello accurato resterebbe confinato al laboratorio.

Capitolo 3

Lo Stack Tecnologico

3.1 Il linguaggio Scala: programmazione funzionale applicata ai Big Data

Scala è un linguaggio multi-paradigma che combina programmazione orientata agli oggetti e programmazione funzionale sulla Java Virtual Machine (JVM). La sua progettazione punta a un *type system* espressivo, a immutabilità per default nelle strutture dati e a interoperabilità diretta con l'ecosistema Java, caratteristiche rilevanti per sistemi distribuiti di medie e grandi dimensioni [21].

Nel contesto di questo lavoro, Scala non è stato scelto per preferenze sintattiche, bensì perché rappresenta il *linguaggio nativo* di Apache Spark. Le API di Spark sono scritte in Scala; le trasformazioni su `Dataset` tipizzati sfruttano *encoder* derivati dal compilatore, evitando la serializzazione Python–JVM di PySpark, che introduce overhead non trascurabile quando il volume di eventi cresce [28].

Il progetto adotta la versione **2.13.18** (`build.sbt`), con prefisso di package `it.utiu.thesis`, in linea con Spark 4.1.1 e con le ottimizzazioni delle collezioni della serie 2.13.

3.1.1 Immutabilità, composizione e type system

Un principio ricorrente nel codice è l'uso di trasformazioni *lazy* su `DataFrame` anziché mutazioni esplicite. In `EcgModelTrainer`, la catena `read` → `na.drop` → `randomSplit` costruisce nuove viste logiche senza modificare il dataset precedente; la pipeline ML concatena `StringIndexer`, `VectorAssembler` e `RandomForestClassifier` in un unico `Pipeline` immutabile.

Le **case class** e il **pattern matching** facilitano la modellazione di strutture dati e la gestione di casi limite (ad esempio, assenza del modello su disco in `AppRunner`) con esaustività verificata dal compilatore. Nel dominio Spark, il vantaggio del type system emerge soprattutto con `Dataset[T]`: gli *implicit encoder* permettono serializzazione efficiente di tipi custom, a differenza di PySpark dove gran parte del percorso resta reflection-based e più costoso su piccoli batch ad alta frequenza.

In un sistema con thread concorrenti (predictor in background, simulatore sul main thread), l'assenza di stato condiviso mutabile tra componenti riduce race condition e semplifica il ragionamento sul comportamento in failure.

3.1.2 Confronto con alternative

PySpark offre una curva di apprendimento più rapida, ma ogni invocazione attraversa Py4J. Con un messaggio al secondo il costo è trascurabile; con centinaia di battiti al minuto per paziente e più stream paralleli, la latenza di bridging diventa misurabile. Java puro manca della concisione per esprimere pipeline dichiarative in poche righe. Scala resta, per questo progetto, il compromesso preferito: runtime JVM, API Spark native, codice batch e streaming nello stesso repository.

3.2 Apache Spark: Core, Spark SQL e MLlib

Apache Spark generalizza MapReduce con operazioni in-memory su strutture resilienti (RDD e, oggi, tabelle SQL) [27], [28]. Il progetto utilizza **Spark 4.1.1** con moduli `spark-core`, `spark-sql`, `spark-mllib` e connettore `spark-sql-kafka-0-10`.

3.2.1 Modello di esecuzione

Spark organizza il lavoro attorno a un *driver* e a uno o più *executor*. Il driver costruisce un DAG di *stage*; ogni stage contiene task parallelizzabili su partizioni di dati. In `Config.scala`, `SPARK_MASTER = "local[*]"` impiega tutti i core della macchina host su un singolo nodo: adeguato alla prototipazione, non rappresentativo di un cluster YARN, Kubernetes o Databricks.

3.2.2 Spark SQL, Catalyst e Tungsten

Spark SQL espone un'API dichiarativa su tabelle strutturate. L'ottimizzatore **Catalyst** [4] trasforma le operazioni attraverso una catena di fasi ben definita:

1. **Parsing**: analisi sintattica della query o del piano DataFrame;
2. **Analysis**: risoluzione di nomi, tipi e vincoli (catalogo);
3. **Logical optimization**: regole di rewrite (pushdown di filtri e proiezioni, pruning di colonne);
4. **Physical planning**: scelta di algoritmi (broadcast join, shuffle hash join, ecc.);
5. **Preparation**: generazione di codice eseguibile.

Nel trainer, operazioni come `groupBy("type").count()` dopo la lettura CSV beneficiano di predicate e projection pushdown senza intervento manuale. Nel predictor, il parsing `from_json`

e la **transform** del modello ML si integrano nello stesso planner, evitando materializzazioni intermedie superflue quando il motore può fonderle.

Tungsten [20] completa Catalyst con esecuzione orientata alla CPU e alla memoria: gestione off-heap, formato binario columnar interno e *whole-stage code generation*, che compila catene di operatori in un unico bytecode JVM. Su micro-batch ripetitivi (filtro, parsing JSON, proiezione colonne output), il codegen riduce il overhead per record rispetto a un interprete riga per riga.

La Figura 3.1 sintetizza il percorso dal codice applicativo all'esecuzione.

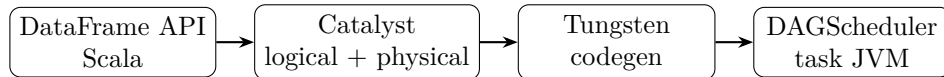


Figura 3.1: Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.

3.2.3 MLlib, Pipeline API e confronto con alternative

MLlib espone algoritmi distribuiti tramite **Estimator**, **Transformer** e **Pipeline** [18]. Salvare `pipelineModel.write().save(...)` persiste in un colpo solo indicizzatore, assemblatore e Random Forest; `PipelineModel.load` garantisce che l'inferenza streaming replichi esattamente le trasformazioni del training.

Rispetto a **scikit-learn**, MLlib scala su cluster e condivide runtime con Structured Streaming, ma offre un catalogo algoritmi più ristretto e meno comodo per esperimenti rapidi su singola macchina. Rispetto a **TensorFlow/PyTorch**, MLlib non punta al deep learning end-to-end: reti neurali richiederebbero export ONNX, serving dedicato o integrazioni non native alla Pipeline API, aumentando il *technical debt* operativo descritto in letteratura per sistemi ML ibridi [25].

XGBoost4J-Spark e **GBTClassifier** (MLlib) sono alternative plausibili per tabular data; Random Forest è stato preferito per integrazione nativa nella Pipeline, assenza di dipendenze aggiuntive e buona gestione di feature eterogenee senza scaling obbligatorio. La limitazione principale resta l'assenza di tuning sistematico (**CrossValidator**) nel prototipo, delegato a estensioni future.

3.2.4 Structured Streaming: semantica temporale e trigger

Structured Streaming tratta il flusso come tabella illimitata aggiornata nel tempo [2]. L'API replica il batch: `readStream`, trasformazioni, `writeStream`.

Processing time ed event time. Per default, il motore ragiona in **processing time** (arrivo dei record al consumer). Con **event time** e **watermark**, Spark gestisce record in ritardo fino a una soglia configurabile; oltre quella soglia, gli eventi tardivi possono essere scartati o inviati a un side output. Nel caso ECG del simulatore, timestamp e ritmo di invio sono allineati: non è stato necessario configurare watermark, ma un Holter reale con buffer offline richiederebbe event time esplicito sul campo temporale del payload.

Trigger. I trigger principali sono: `ProcessingTime(interval)` (micro-batch periodici), `Once / AvailableNow` (elaborazione fino a fine log, utile per catch-up), e il default *as fast as possible* adottato dal predictor, che avvia un micro-batch non appena sono disponibili dati da Kafka.

Micro-batch vs continuous processing. Structured Streaming implementa, di fatto, micro-batch su Spark SQL; la modalità *continuous processing* (latenza sub-secondo) resta sperimentale e limitata nel set di operatori. Per la tesi, il modello a micro-batch è accettabile: latenza end-to-end nell'ordine di centinaia di millisecondi resta compatibile con monitoraggio *near-real-time*, come discusso nel Capitolo 6.

Nel componente `EcgStreamingPredictor`, Structured Streaming consuma Kafka, applica `pipelineModel.transform` e scrive su Delta tramite `foreachBatch`. Questa scelta consente logica custom (`persist`, `isEmpty`, logging) a fronte di una gestione più esplicita del sink rispetto a `format("delta")` nativo.

3.3 Apache Kafka: architettura publish-subscribe e disaccoppiamento

Apache Kafka organizza la messaggistica attorno a *topic*, *partizioni* e *offset* [12], [14]. I producer appendono record immutabili; i consumer leggono tramite *consumer group* con bilanciamento del carico.

3.3.1 Architettura log-centrica

Ogni topic è un log replicato e partizionato, non una coda che consuma e dimentica. Più consumer possono rileggere lo stesso flusso a offset diversi (inferenza real-time, job di riprocessamento batch, audit).

Nel progetto, `ecg-input-stream` bufferizza tra `EcgPatientSimulator` e `EcgStreamingPredictor`: il producer può inviare un battito al secondo mentre Spark elabora micro-batch con latenza variabile, senza vincolare le due velocità.

3.3.2 Partizioni, consumer group e parallelismo

Una **partizione** è l'unità di ordinamento e parallelismo: i record con la stessa chiave finiscono nella stessa partizione, preservando l'ordine per chiave. Il numero massimo di consumer attivi in un gruppo che leggono in parallelo un topic è, in prima approssimazione, il numero di partizioni del topic. Con una sola partizione (default in sviluppo), il throughput di consumo Spark resta limitato a un task per micro-batch; aumentare le partizioni e `spark.default.parallelism` è il primo passo verso scaling orizzontale.

Structured Streaming gestisce internamente gli offset Kafka nel checkpoint: al restart, riprende dall'ultimo offset committato insieme al batch Delta, non dal solo puntatore del consumer Java.

3.3.3 Modalità KRaft e deprecazione di ZooKeeper

Storicamente, Kafka usava ZooKeeper per metadati di cluster. KRaft (Kafka Raft) introduce un quorum interno [1], riducendo componenti da operare.

Il `docker-compose.yml` del repository configura un nodo combinato broker+controller, quorum `1@localhost:9093`, `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1`. In produzione, broker e controller andrebbero separati con replication factor ≥ 3 .

Tabella 3.1: Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.

Aspetto	ZooKeeper	KRaft
Componenti esterni	Sì (cluster ZK separato)	No
Tempo di failover	Secondi–decine di s	Sub-secondo (target)
Complessità operativa	Alta	Ridotta
Scalabilità metadata	Limitata	Migliorata

3.3.4 Garanzie di consegna ed exactly-once

Kafka supporta:

- **at-most-once**: messaggi possono perdersi, nessun duplicato;
- **at-least-once**: nessuna perdita, possibili duplicati al retry;
- **exactly-once** (EOS): producer idempotente + transazioni quando producer e consumer partecipano allo stesso cluster transazionale.

Spark Structured Streaming, con checkpoint e sink idempotente, punta a **exactly-once a livello di micro-batch**: ogni batch è processato una volta anche dopo restart del driver, a patto che la scrittura su Delta e il commit degli offset avvengano nella stessa transazione logica del batch. Duplicati a livello di *business key* (**record**) restano possibili se si riesegue l'intero esperimento senza deduplicazione: argomento ripreso nel Capitolo 4.

3.4 Delta Lake: il paradigma Data Lakehouse

Delta Lake estende Parquet con un *transaction log* che garantisce proprietà ACID su object store e filesystem [3]. Il paradigma *Lakehouse* unifica flessibilità del data lake e affidabilità del warehouse.

3.4.1 Limiti di Parquet puro

Parquet da solo non gestisce scritture concorrenti né versioning: due writer che appendono possono corrompere la directory. Per predizioni cliniche, dove ogni battito classificato deve essere persistito in modo auditabile, queste lacune sono critiche.

3.4.2 Transaction log, ACID e schema

Delta mantiene `_delta_log` con operazioni append, overwrite, merge, delete. Le scritture sono atomiche; l'*optimistic concurrency control* risolve conflitti tra writer. Per default vale **schema enforcement**: un append con colonne incompatibili fallisce. Con `mergeSchema=true` (schema *evolution*), nuove colonne possono essere aggiunte; utile se il payload JSON guadagna attributi, ma richiede allineamento anche sul parser `from_json` a monte.

3.4.3 Time travel, VACUUM e audit clinico

Time travel permette letture per versione (`VERSION AS OF`) o timestamp (`TIMESTAMP AS OF`), ricostruendo lo stato della tabella a un istante passato. In contesti regolati, ciò supporta audit: quale versione del log predittivo era visibile quando è stato generato un alert.

VACUUM rimuove file dati non referenziati dal log oltre una retention configurabile (default 7 giorni), bilanciando spazio disco e possibilità di rollback. Policy troppo aggressive possono impedire time travel su versioni antiche: scelta da calibrare con requisiti di conservazione clinica.

3.4.4 Integrazione con Spark

Il progetto registra le estensioni Delta nella `SparkSession`:

Listing 3.1: Configurazione Delta Lake in `SparkSession`.

```
1 .config("spark.sql.extensions",  
2   "io.delta.sql.DeltaSparkSessionExtension")  
3 .config("spark.sql.catalog.spark_catalog",  
4   "org.apache.spark.sql.delta.catalog.DeltaCatalog")
```

La stessa configurazione compare in `EcgModelTrainer` e `EcgStreamingPredictor`; solo il predictor scrive su `spark-warehouse/ecg_predictions`. Il checkpoint Spark risiede in `spark-warehouse/checkpoints/ecg_predictions`: separare metadati di offset da dati di business è buona pratica operativa.

3.5 Sintesi: requisiti e tecnologie adottate

La Tabella 3.2 collega i requisiti dell'introduzione alle scelte tecnologiche.

Tabella 3.2: Mappatura requisiti–tecnologie.

Requisito	Tecnologia	Meccanismo
Disaccoppiamento ingestio- n/processing	Kafka	Topic <code>ecg-input-stream</code> , parti- zioni, offset
Training su dati storici	Spark MLlib	<code>EcgModelTrainer</code> , split 80/20, <code>Pipeline.fit</code>

Tabella 3.2 — *continua*

Requisito	Tecnologia	Meccanismo
Inferenza near-real-time	Structured Streaming	Micro-batch da Kafka, <code>pipelineModel.transform</code>
Persistenza affidabile	Delta Lake	Append transazionale, time travel, <code>foreachBatch</code>
Fault tolerance	Checkpoint Spark	<code>STREAMING_CHECKPOINT_PATH</code>
Linguaggio unificato	Scala 2.13	Batch e streaming nello stesso codebase

Nel capitolo seguente queste tecnologie sono organizzate in moduli e flussi dati concreti, orchestrati da **AppRunner**.

Capitolo 4

Progettazione e Architettura del Sistema

4.1 Pattern architetturali: Lambda, Kappa e posizionamento del progetto

Le architetture per l'elaborazione di dati in tempo reale si sono consolidate attorno a due modelli di riferimento [12], [13], [17].

L'**architettura Lambda** prevede due pipeline parallele: un *batch layer* che calcola viste corrette su dati storici e uno *speed layer* che gestisce gli aggiornamenti recenti con latenza ridotta; un *serving layer* unifica i risultati per le query. La forza della Lambda è la correttezza batch su volumi elevati; il costo è la duplicazione della logica (stesse aggregazioni implementate due volte) e la complessità operativa.

L'**architettura Kappa** propone un unico flusso di eventi: lo speed layer consuma il log; il riprocessamento storico si ottiene rileggendo il topic dall'inizio con job batch-equivalenti. Si elimina la duplicazione logica, ma servono formati immutabili, retention adeguata e capacità di ricalcolo su scala. La Figura 4.1 e la Tabella 4.1 riassumono la Lambda e il confronto con l'impostazione adottata.

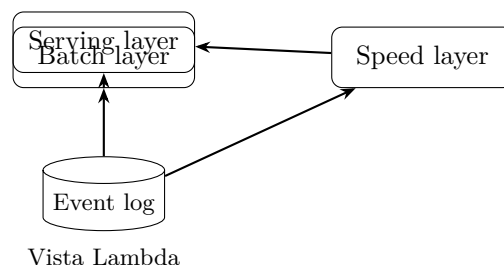


Figura 4.1: Schema concettuale dell'architettura Lambda: batch e speed layer alimentano il serving.

Il progetto di questa tesi adotta un **ibrido pragmatico**. L'addestramento avviene in batch off-line (`EcgModelTrainer`); l'inferenza opera in streaming (`EcgStreamingPredictor`). Non esiste uno speed layer che ricalcoli aggregati su finestra mobile: il modello è un artefatto statico ricaricato dal filesystem fino a un nuovo training. La configurazione è vicina alla Lambda con confine netto training/serving, pratica comune nei sistemi ML operativi dove la frequenza di retraining è ordini di grandezza inferiore a quella di inferenza [25].

Tabella 4.1: Confronto sintetico Lambda, Kappa e impostazione del progetto.

Aspetto	Lambda	Kappa	Questo progetto
Logica di calcolo	Duplicata (batch + speed)	Unificata sul log	Train vs infer separati
Latenza inferenza	Bassa (speed layer)	Bassa	Bassa (streaming)
Correttezza storica	Alta (batch)	Ricalcolo dal log	Training batch su CSV
Complessità ops	Alta	Media	Bassa (prototipo)
Model serving	Non specificato	Non specificato	Artefatto ML su FS

Rispetto alla Kappa pura, manca la possibilità di riaddestrare rileggendo soltanto Kafka: il training legge CSV storici, non il topic. Rispetto alla Lambda pura, manca un serving layer che fondo viste batch e real-time; le predizioni sono scritte direttamente su Delta Lake dal predictor.

4.2 Panoramica della pipeline

La pipeline si articola in tre moduli funzionali e un orchestratore:

1. **Batch layer** — `EcgModelTrainer`: legge CSV storici, addestra la pipeline ML, salva il modello su filesystem.
2. **Ingestion layer** — `EcgPatientSimulator`: simula un dispositivo edge, pubblica battiti JSON su Kafka.
3. **Streaming layer** — `EcgStreamingPredictor`: consuma Kafka, classifica, persiste su Delta Lake.
4. **Orchestratore** — `AppRunner`: coordina l'avvio sequenziale dei componenti.

La Figura 4.2 mostra il flusso logico dei dati; la Figura 4.3 ne dettaglia la sequenza temporale di avvio.

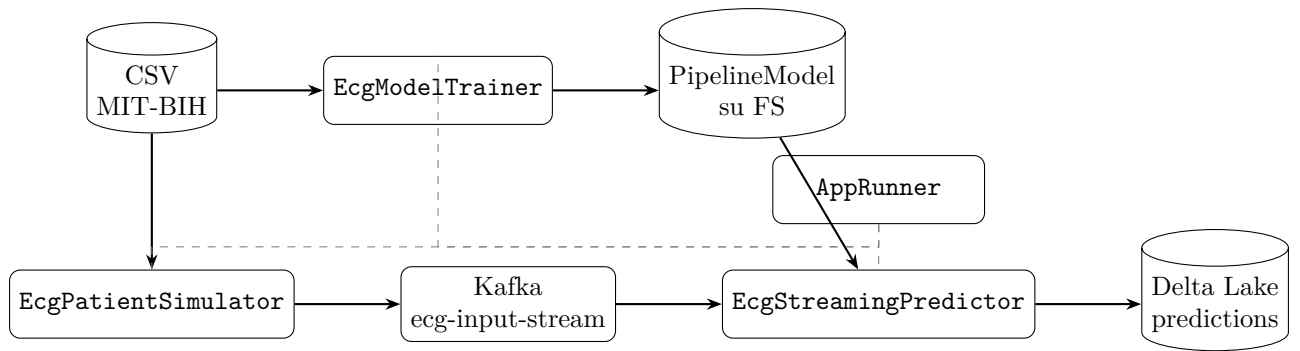


Figura 4.2: Flusso logico: training batch, ingestione simulata e inferenza streaming. Linee tratteggiate: orchestrazione **AppRunner**.

4.2.1 Flusso granulare dei dati

In sintesi: il trainer costruisce il modello da CSV; il simulatore pubblica JSON su Kafka; il predictor deserializza, classifica e appende su Delta con checkpoint. Il dettaglio implementativo (listing, `foreachBatch`, configurazione) è nel Capitolo 5. Il modello ML (`PipelineModel.load`) è l'unico artefatto condiviso tra batch e streaming.

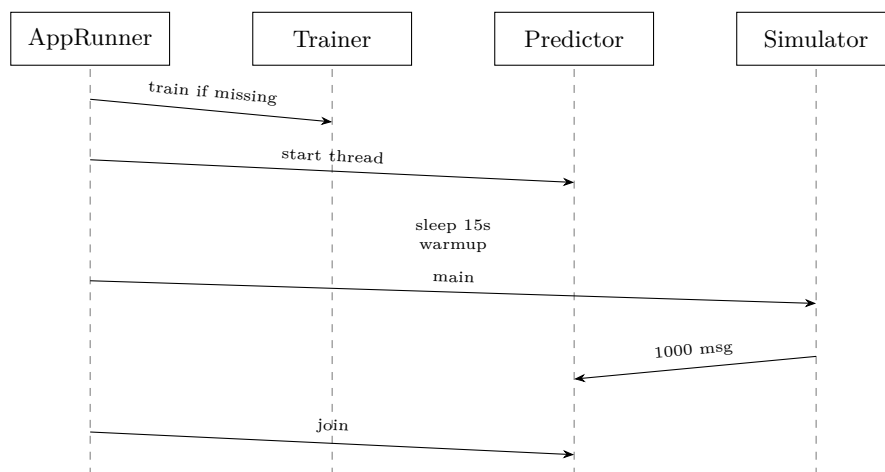


Figura 4.3: Sequenza di avvio orchestrata da **AppRunner**: training condizionale, predictor in thread demone, attesa, simulatore, join.

4.3 Vista fisica e deployment

In ambiente sperimentale, l'intera pipeline gira su un singolo host di sviluppo. La Figura 4.4 illustra la disposizione fisica.

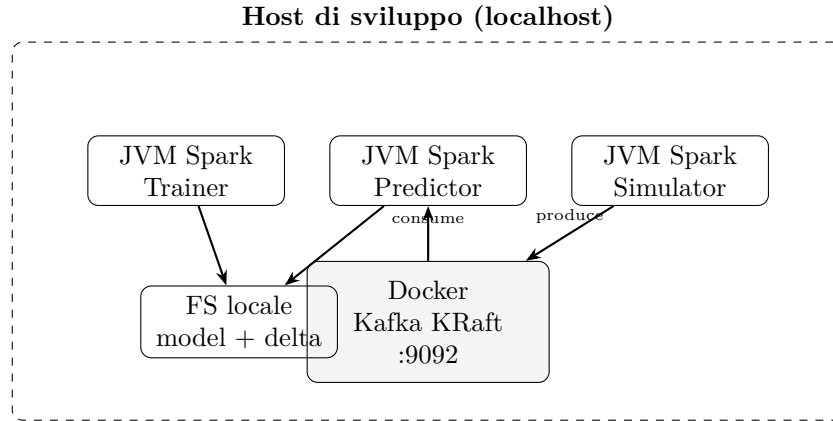


Figura 4.4: Deployment mono-nodo: tre processi JVM Spark (`local[*]`) e un container Docker per Kafka.

4.3.1 Limiti del setup mono-nodo

`SPARK_MASTER = "local[*]"` impiega tutti i core su un'unica macchina; Kafka ha replication factor 1. Un crash del host interrompe l'intera pipeline. In produzione: cluster Spark dedicato, Kafka multi-broker con $RF \geq 3$, Delta su object store (S3, ADLS). Il setup attuale è deliberatamente minimalista per dimostrare l'integrazione end-to-end.

4.4 Fase di addestramento: `EcgModelTrainer`

Il modulo batch implementa il ciclo completo di machine learning supervisionato.

4.4.1 Flusso operativo

1. Lettura CSV da `DATASET_PATH` con header e inferenza schema.
2. Pulizia: `na.drop()` elimina righe con valori mancanti.
3. Split train/test 80/20 con `seed = 42L` per riproducibilità.
4. Costruzione pipeline: `StringIndexer` $\rightarrow$ `VectorAssembler` $\rightarrow$ `RandomForestClassifier`.
5. Addestramento (`pipeline.fit`), valutazione su test set, salvataggio su `ML_MODEL_PATH`.

4.4.2 Scelta del classificatore e alternative

Random Forest è configurato con `numTrees=100` e `maxDepth=15` [6]. La scelta risponde alla natura tabellare delle feature (intervalli `pre-RR/post-RR`, morfologia QRS e intervalli PQ/QT/ST su due derivazioni) e alla capacità degli ensemble di catturare interazioni non lineari senza normalizzazione rigorosa.

Alternative valutabili nello stesso stack Spark includono:

- **GBTCClassifier** (gradient boosting nativo MLlib): spesso competitivo su dati tabellari, con tempi di training superiori e maggiore rischio di overfitting senza tuning;
- **LinearSVC / LogisticRegression**: baseline interpretabili, meno adatte se le interazioni tra feature RR e morfologia QRS sono forti;
- **Reti neurali**: richiederebbero export e serving fuori dalla Pipeline MLlib, aumentando la complessità operativa senza beneficio dimostrato su questo dataset nel prototipo.

Feature importance. Random Forest fornisce, per ogni albero, una misura di impurezza ridotta per split; aggregando sui 100 alberi si ottiene un ranking delle feature utile per dialogare con il dominio clinico (quali descrittori RR o QRS guidano le confusions tra N e V). Non è implementata nel codice attuale, ma resta un'estensione naturale per migliorare l'interpretabilità senza cambiare architettura.

Cross-validation. MLlib espone `CrossValidator` e `TrainValidationSplit` per grid search su `numTrees` e `maxDepth`. Non sono stati usati: il focus della tesi è l'integrazione streaming, e il dataset preprocessato ha dimensione moderata; un tuning sistematico migliorerebbe la generalizzazione a costo di tempi di sperimentazione molto più lunghi.

4.4.3 Output del trainer

Il modello salvato include metadata, stages serializzati e statistiche dell'indicizzatore. In streaming, etichette mai viste in training sono gestite da `setHandleInvalid("keep")`, che assegna un indice dedicato anziché sollevare eccezioni: robustezza operativa a scapito di predizioni potenzialmente poco informative su classi *unknown*.

4.5 Simulazione IoT: EcgPatientSimulator

Il simulatore riproduce un dispositivo wearable che trasmette battiti verso un broker centralizzato.

4.5.1 Logica di produzione

1. Lettura di al massimo 1000 record dal CSV (`.limit(1000)`).
2. Serializzazione JSON tramite `to_json(struct(col("*")))`.
3. Raccolta in memoria (`.collect()`) e invio sequenziale con `KafkaProducer`.
4. Intervallo di un secondo tra messaggi (`Thread.sleep(1000)`).

L'uso di `KafkaProducer` Java puro, anziché `df.writeStream.format("kafka")`, avvicina il simulatore a un producer esterno a Spark, come avverrebbe su firmware edge senza motore distribuito.

4.5.2 Schema del payload e impatto sulla banda

Ogni messaggio serializza *tutte* le colonne del CSV: identificativo **record**, etichetta **type**, feature numeriche e metadati. La chiave **beat-{index}** serve al debug ma non è sfruttata per partizionamento semantico (paziente, sessione). In produzione, si potrebbe ridurre il payload inviando solo le feature richieste dal modello, con risparmio di banda e minor superficie di esposizione dati; il contratto andrebbe però versionato esplicitamente (Schema Registry, vedi §4.6.2).

4.5.3 Estensioni e limiti

Il rate fisso di 1 msg/s è una semplificazione rispetto a un Holter reale (jitter, burst dopo riconnessione, gap di trasmissione). Estensioni naturali: intervallo casuale gaussiano intorno alla frequenza cardiaca; simulazione di disconnessioni; ripresa con **startingOffsets** e idempotenza lato consumer.

`.collect()` materializza 1000 record sul driver: accettabile in demo, inaccettabile per flussi continui. Questi limiti sono consapevoli e discussi nelle conclusioni.

4.6 Inferenza in tempo reale: EcgStreamingPredictor

Il predictor è il cuore operativo della pipeline in fase di serving.

4.6.1 Bootstrap dello schema JSON

Aniché hardcodare uno **StructType**, il codice legge lo schema dal CSV di training:

Listing 4.1: Inferenza schema da CSV per `from_json`.

```
1 val jsonSchema = spark.read
2   .option("header", "true")
3   .option("inferSchema", "true")
4   .csv(datasetPath)
5   .schema
```

Vantaggio: nessuna duplicazione manuale tra producer e consumer. Svantaggio: accoppiamento implicito al layout del CSV; una modifica delle colonne richiede riavvio coerente di simulatore e predictor.

4.6.2 Schema Registry e contratto dati

In ambienti industriali, **Confluent Schema Registry** (o equivalenti) gestisce schemi Avro, JSON Schema o Protobuf con regole di compatibilità (*backward*, *forward*). Il producer registra la versione *v* del messaggio; il consumer la risolve a runtime, fallendo in modo controllato se il contratto non è compatibile. Per il prototipo accademico, l'inferenza da CSV è sufficiente;

per produzione clinica, un contratto esplicito riduce il rischio di parsing silenzioso con campi mancanti o tipi errati dopo aggiornamento firmware del dispositivo.

4.6.3 Pipeline di inferenza

1. `readStream.format("kafka")` con `startingOffsets=earliest` (solo al primo avvio senza checkpoint).
2. Cast del payload a stringa, parsing con `from_json`.
3. Trasformazione tramite `pipelineModel.transform`.
4. Proiezione su `record`, `true_label`, `prediction`, `probability`.
5. Scrittura su Delta Lake via `foreachBatch` con checkpoint.

4.6.4 Ciclo di vita del modello in produzione

Il pattern attuale — modello su directory locale, caricato all'avvio del predictor — è tipico dei prototipi ma insufficiente per operazioni mature [25]:

- **Model serving:** registry (MLflow, servizio REST) con versioni immutabili e metadati di training;
- **A/B testing:** routing di una frazione del traffico su modello candidato;
- **Model drift:** monitoraggio distribuzione feature e calo di metriche, che innesci retraining;
- **Hot-swap:** sostituzione del modello senza fermare il job streaming (richiede doppio caricamento o query restart controllato).

Nessuno di questi meccanismi è implementato; il trainer sovrascrive `ML_MODEL_PATH` e il predictor deve essere riavviato per caricare una nuova versione.

4.6.5 Fault tolerance e exactly-once

Il checkpoint in `STREAMING_CHECKPOINT_PATH` registra offset Kafka e metadati del query plan. In caso di failure del driver, il restart riprende dall'ultimo checkpoint committato. Checkpoint + append idempotente su Delta garantisce exactly-once a livello di micro-batch; duplicati di business key restano possibili se si ripete l'esperimento senza deduplicazione.

4.7 Orchestrazione: AppRunner

AppRunner coordina l'avvio in sequenza logica:

1. Verifica esistenza del modello in `ML_MODEL_PATH`; se assente, invoca `EcgModelTrainer.main`.
2. Avvia `EcgStreamingPredictor` in un thread demone (`setDaemon(true)`).
3. Attende 15 secondi (`Thread.sleep(15000)`) per il warmup di Spark Streaming.
4. Avvia `EcgPatientSimulator`.
5. Attende la terminazione del thread predictor (`predictorThread.join()`).

L'attesa di 15 secondi è un'euristica che evita che i primi messaggi arrivino prima che la query streaming sia attiva; in produzione andrebbe sostituita da health-check (`isActive`, connettività Kafka, readiness del consumer group).

4.8 Requisiti non funzionali

4.8.1 Disaccoppiamento

Kafka isola producer e consumer: il simulatore può terminare dopo 1000 messaggi mentre il predictor consuma il backlog. Il trainer batch è indipendente dal flusso streaming, eseguibile on-demand o schedulato (cron, Airflow).

4.8.2 Idempotenza e consistenza

Delta Lake append con checkpoint Spark produce un log immutabile di predizioni. Ripetere l'esperimento con gli stessi dati genera righe duplicate (non idempotenza su `record`); un `MERGE` Delta su chiave risolverebbe il problema.

4.8.3 Scalabilità

La scalabilità orizzontale passa dal parallelismo delle partizioni Kafka e dal numero di executor Spark. Con topic a partizione singola, il consumo resta serializzato; aumentare partizioni e parallelismo sblocca throughput superiore, come stimato nel Capitolo 6.

4.8.4 Evolvibilità dello schema

L'approccio JSON implicito è fragile: Delta può evolvere lo schema in append, ma `from_json` fallisce su tipi incompatibili o campi obbligatori mancanti. Formati schema-first (Avro, Protobuf) e Registry mitigano il rischio.

4.8.5 Observability

Il prototipo si affida a log su console (`println`, `show` nei batch) e alla Spark UI locale. In produzione servirebbero metriche esportate (throughput consumer, lag Kafka, latenza micro-batch, error rate parsing) verso Prometheus/Grafana o equivalenti cloud, con tracing distribuito per correlare `batchId` e offset. Senza observability strutturata, diagnosticare colli di bottiglia o drift operativo resta difficile oltre la demo controllata.

4.8.6 Sicurezza

L'istanza Kafka in Docker espone PLAINTEXT su localhost: nessuna autenticazione SASL, nessun TLS. I dati ECG, anche come feature, sono informazioni sanitarie sensibili: un deployment reale richiederebbe cifratura in transito (TLS tra edge, broker e Spark), controllo accessi al topic, cifratura at-rest su Delta e policy di retention allineate al GDPR. Il prototipo accademico omette questi controlli per ridurre la frizione sperimentale; vanno elencati esplicitamente come gap verso un sistema clinicamente deployabile.

Il Capitolo 5 approfondisce configurazione e codice; il Capitolo 6 raccoglie le misure sperimentali.

Capitolo 5

Implementazione e Dettagli Tecnici

5.1 Organizzazione del codice sorgente

Il repository `ecg-streaming-classifier` segue una struttura piatta, coerente con la dimensione del progetto:

```
src/main/scala/  
  AppRunner.scala  
  batch/EcgModelTrainer.scala  
  streaming/EcgPatientSimulator.scala  
  streaming/EcgStreamingPredictor.scala  
  utils/Config.scala
```

Il package root è `it.utiu.thesis`, definito in `build.sbt` tramite `idePackagePrefix`. I sottopackage `batch`, `streaming` e `utils` separano responsabilità senza introdurre layer di astrazione superflui.

5.1.1 Centralizzazione della configurazione

Tutti i parametri operativi risiedono in `utils.Config`:

Listing 5.1: Parametri centralizzati in `Config.scala`.

```
1 object Config {  
2   val KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"  
3   val KAFKA_TOPIC = "ecg-input-stream"  
4   val SPARK_MASTER = "local[*]"  
5  
6   val DATASET_PATH = "/home/andrea/data/ecg/*.csv"  
7   val ML_MODEL_PATH = "ml-model/ecg-pipeline-model"  
8  
9   val DELTA_PREDICTIONS_PATH = "spark-warehouse/ecg_predictions"  
10  val STREAMING_CHECKPOINT_PATH =  
11    "spark-warehouse/checkpoints/ecg_predictions"  
12 }
```


La configurazione hardcoded è un limite riconosciuto del prototipo. In un'evoluzione produttiva, i path e gli endpoint andrebbero esternalizzati tramite Typesafe Config (HOCON), variabili d'ambiente o un servizio di configurazione centralizzato. Il path del dataset punta a un percorso assoluto Linux (`/home/andrea/data/ecg/*.csv`): su Windows o macOS va adattato manualmente prima dell'esecuzione.

5.2 Build e gestione delle dipendenze

Il progetto è gestito con **SBT 1.12.4** e Scala **2.13.18**. Il file `build.sbt` fissa le versioni dello stack analitico:

Listing 5.2: Dipendenze principali in `build.sbt`.

```
1 val sparkVersion = "4.1.1"
2 val deltaVersion = "4.1.0"
3
4 libraryDependencies += Seq(
5   "org.apache.spark" %% "spark-core" % sparkVersion,
6   "org.apache.spark" %% "spark-sql" % sparkVersion,
7   "org.apache.spark" %% "spark-mllib" % sparkVersion,
8   "org.apache.spark" %% "spark-sql-kafka-0-10" % sparkVersion,
9   "io.delta" %% "delta-spark" % deltaVersion,
10 )
```

Spark e Delta condividono la stessa release train (4.1.x): questo riduce incompatibilità tra catalogo Delta e runtime SQL. Il connettore `spark-sql-kafka-0-10` è allineato alla versione Spark, requisito per Structured Streaming su Kafka. Non sono presenti dipendenze di test (`scalatest`) né plugin di packaging (`sbt-assembly`): coerente con un prototipo didattico, ma limitante per CI automatizzata.

`idePackagePrefix := Some("it.utiu.thesis")` imposta il package root per l'IDE senza alterare la struttura delle directory sotto `src/main/scala/`.

5.3 Infrastruttura Docker e broker Kafka

Kafka è avviato tramite `docker-compose.yml` con immagine `apache/kafka:latest` in modalità **KRaft** su un singolo nodo (`broker,controller`). I parametri salienti sono: `KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093`, `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1`, listener `PLAINTEXT` su porta 9092.

In laboratorio questo basta a esercitare producer e consumer locali. In produzione servirebbero almeno tre broker, replication factor ≥ 3 e separazione dei ruoli controller/broker; altrimenti un guasto al container interrompe l'intera ingestion.

Spark non è containerizzato: gira sulla JVM host con `local[*]`. La scelta semplifica il debug (log unificati, Spark UI su 4040) a scapito del disaccoppiamento infrastrutturale tipico dei deployment cloud-native.

5.4 Configurazione SparkSession

Trainer, predictor e simulatore creano ciascuno una propria `SparkSession`, ripetendo la stessa configurazione Delta:

Listing 5.3: Configurazione comune della sessione Spark.

```
1 SparkSession.builder()
2   .appName("EcgModelTrainer") // o EcgStreamingPredictor / EcgPatientSimulator
3   .master(Config.SPARK_MASTER)
4   .config("spark.driver.host", "127.0.0.1")
5   .config("spark.driver.bindAddress", "127.0.0.1")
6   .config("spark.sql.extensions",
7     "io.delta.sql.DeltaSparkSessionExtension")
8   .config("spark.sql.catalog.spark_catalog",
9     "org.apache.spark.sql.delta.catalog.DeltaCatalog")
10  .getOrCreate()
```

`SPARK_MASTER = "local[*]"` assegna tutti i core della macchina a un unico executor logico: adeguato per ~100 000 righe CSV e micro-batch piccoli, insufficiente per carichi multi-paziente senza cluster.

Parametri di tuning non sono stati variati (`spark.sql.shuffle.partitions`, memoria driver/executor): su dataset tabellari di questa dimensione i default Spark 4.x sono generalmente sufficienti; su cluster andrebbero calibrati in funzione del numero di partizioni Kafka e della cardinalità del modello.

5.5 Data pipeline e machine learning

5.5.1 Preprocessing e feature engineering

Il trainer legge CSV con header e inferenza automatica dello schema. La pulizia è minimale: `rawDF.na.drop()` elimina righe con qualsiasi valore nullo, senza imputazione.

La selezione delle colonne feature avviene per esclusione:

Listing 5.4: Selezione colonne feature in `EcgModelTrainer`.

```
1 val featureCols = cleanDF.columns.filter(colName =>
2   colName != "record" && colName != "type" && colName != "label"
3 )
```

Le colonne `record` (identificativo traccia) e `type` (etichetta testuale) vengono escluse dal vettore numerico. L'esclusione di `label` è difensiva: evita leakage se la colonna fosse presente nel CSV con valori numerici già indicizzati.

Lo `StringIndexer` converte `type` in `label` numerico:

Listing 5.5: Pipeline ML: indicizzazione, assemblaggio, classificazione.

```

1  val indexer = new StringIndexer()
2    .setInputCol("type")
3    .setOutputCol("label")
4    .setHandleInvalid("keep")
5
6  val assembler = new VectorAssembler()
7    .setInputCols(featureCols)
8    .setOutputCol("features")
9    .setHandleInvalid("keep")
10
11 val rf = new RandomForestClassifier()
12   .setLabelCol("label")
13   .setFeaturesCol("features")
14   .setNumTrees(100)
15   .setMaxDepth(15)
16
17 val pipeline = new Pipeline()
18   .setStages(Array(indexer, assembler, rf))

```

`setHandleInvalid("keep")` su entrambi gli stadi è una scelta pragmatica. In training, etichette o feature mancanti ricevono un bucket dedicato anziché causare eccezioni. In streaming, battiti con feature incomplete possono comunque transitare nella pipeline, producendo predizioni potenzialmente inaffidabili. Un approccio più rigoroso prevedrebbe filtraggio a monte (**drop** o quarantena su topic separato) anziché tolleranza silenziosa.

5.5.2 Split, addestramento e valutazione

Listing 5.6: Split train/test e valutazione.

```

1  val Array(trainingData, testData) =
2    cleanDF.randomSplit(Array(0.8, 0.2), seed = 42L)
3
4  val pipelineModel = pipeline.fit(trainingData)
5
6  val predictions = pipelineModel.transform(testData)
7
8  val evaluator = new MulticlassClassificationEvaluator()
9    .setLabelCol("label")
10   .setPredictionCol("prediction")
11   .setMetricName("accuracy")
12
13 val accuracy = evaluator.evaluate(predictions)

```

Lo `seed = 42L` garantisce riproducibilità dello split. L'evaluator calcola solo l'accuratezza globale; metriche per-classe (precision, recall, F1) richiederebbero un `MulticlassClassificationEvaluator` per ogni metrica o l'uso di `BinaryClassificationMetrics` su coppie one-vs-rest. Il Capitolo 6 estende l'analisi con metriche più granulari.

5.5.3 Iperparametri Random Forest

`numTrees=100` e `maxDepth=15` non sono stati ottimizzati con grid search. La scelta riflette un compromesso documentato in letteratura [6]: 100 alberi offrono varianza sufficientemente bassa senza costi eccessivi; profondità 15 limita overfitting su dataset di dimensione moderata. Un tuning

sistematico con `CrossValidator` o `TrainValidationSplit` migliorerebbe la generalizzazione, a costo di tempi di training più lunghi.

5.5.4 Serializzazione e persistenza del modello

Listing 5.7: Salvataggio pipeline su filesystem.

```
1 pipelineModel.write.overwrite().save(modelPath)
```

Spark ML salva la pipeline in una directory con struttura:

```
ml-model/ecg-pipeline-model/  
  metadata/  
  stages/  
    0_StringIndexer_.../  
    1_VectorAssembler_.../  
    2_RandomForestClassificationModel_.../
```

Il flag `overwrite()` consente retraining senza rimozione manuale della directory precedente. In produzione, versionare i modelli (MLflow, DVC) eviterebbe sovrascritture accidentali e permetterebbe rollback.

5.6 Gestione dello streaming

5.6.1 Parsing JSON e schema dinamico

Il consumer Kafka decodifica il payload in due passaggi:

Listing 5.8: Parsing messaggi Kafka in `EcgStreamingPredictor`.

```
1 val kafkaStreamDF = spark.readStream  
2   .format("kafka")  
3   .option("kafka.bootstrap.servers", Config.KAFKA_BOOTSTRAP_SERVERS)  
4   .option("subscribe", Config.KAFKA_TOPIC)  
5   .option("startingOffsets", "earliest")  
6   .load()  
7  
8 val parsedStreamDF = kafkaStreamDF  
9   .selectExpr("CAST(value AS STRING) as json_string")  
10  .select(from_json(col("json_string"), jsonSchema).alias("data"))  
11  .select("data.*")
```

`startingOffsets=earliest` garantisce che, al primo avvio, vengano consumati tutti i messaggi presenti nel topic. In restart con checkpoint, Spark riprende dall'ultimo offset committato, ignorando questa opzione.

L'alternativa professionale — schema Avro registrato su Confluent Schema Registry — offrirebbe validazione strict, compatibilità backward/forward e evoluzione controllata. Per un

prototipo accademico, l'inferenza schema da CSV riduce il boilerplate a costo di accoppiamento implicito.

5.6.2 Trasformazione e proiezione output

Listing 5.9: Inferenza e selezione colonne output.

```

1 val predictionsStreamDF = pipelineModel.transform(parsedStreamDF)
2
3 val finalStreamDF = predictionsStreamDF.select(
4     col("record"),
5     col("type").alias("true_label"),
6     col("prediction"),
7     col("probability")
8 )

```

La colonna `probability` contiene un vettore sparse di probabilità per classe. In analisi successive, estrarre la probabilità della classe predetta richiede UDF o parsing del vettore.

5.6.3 Checkpointing e fault tolerance

L'Algoritmo 1 descrive la logica di `foreachBatch`.

Algorithm 1: Elaborazione micro-batch con `foreachBatch`

Input: Micro-batch B_i , identificativo $batchId$, path Delta P , checkpoint C

Output: Predizioni persistite, offset committati

```

1  $B_i$ .persist();
2 if  $B_i$  non è vuoto then
3     Logga  $batchId$  e mostra prime 20 righe;
4      $B_i$ .write.format("delta").mode("append").save( $P$ );
5 end
6  $B_i$ .unpersist();
7 Spark committa offset in  $C$ ;

```

Listing 5.10: Implementazione `foreachBatch` con Delta Lake.

```

1 val query = finalStreamDF.writeStream
2   .outputMode("append")
3   .option("checkpointLocation", checkpointPath)
4   .foreachBatch { (batchDF: DataFrame, batchId: Long) =>
5     batchDF.persist()
6     if (!batchDF.isEmpty) {
7       batchDF.show(20, truncate = false)
8       batchDF.write.format("delta").mode("append").save(deltaPath)
9     }
10    batchDF.unpersist()
11    ()
12  }
13  .start()

```

`persist()` evita ricalcolo del batch tra `isEmpty`, `show` e `write`. Senza `persist`, Spark potrebbe rieseguire la catena di trasformazioni tre volte, triplicando il costo computazionale.

`outputMode("append")` è l'unico mode compatibile con `foreachBatch` quando il sink non è nativo Spark Streaming. Exactly-once a livello di micro-batch si ottiene dalla combinazione:

- checkpoint che registra offset Kafka processati;
- scrittura Delta atomica per batch;
- restart che riprende dall'ultimo checkpoint senza riprocessare batch già committati.

5.6.4 `foreachBatch` rispetto al sink Delta nativo

Structured Streaming supporta anche:

Listing 5.11: Alternativa dichiarativa (non usata nel prototipo).

```
1 finalStreamDF.writeStream
2   .format("delta")
3   .outputMode("append")
4   .option("checkpointLocation", checkpointPath)
5   .start(deltaPath)
```

Nel progetto si è scelto `foreachBatch` per poter eseguire `persist`, `isEmpty`, `show` e logging condizionale prima della scrittura. Il costo è codice imperativo aggiuntivo e responsabilità di gestione errori spostata sull'applicazione: un'eccezione nel batch non committato non avanza gli offset Kafka.

Il sink nativo Delta è preferibile quando la logica per batch è banale (append puro) e si vuole delegare a Spark la massima ottimizzazione del writer. Per audit clinico con campionamento delle predizioni in console, `foreachBatch` resta la scelta più trasparente in fase di prototipazione.

5.7 Storage e sink: Delta Lake

5.7.1 Configurazione sink

I path di output sono definiti in `Config`: `DELTA_PREDICTIONS_PATH` per i dati di business, `STREAMING_CHECKPOINT_PATH` per i metadati Spark. La separazione è fondamentale: cancellare il checkpoint per "resettare" l'esperimento non deve cancellare le predizioni accumulate.

5.7.2 Operazioni successive

Delta Lake supporta operazioni avanzate non implementate nel prototipo ma rilevanti per un deployment clinico:

- OPTIMIZE — compattazione file small per query più efficienti;
- Z-ORDER BY record — clustering per accesso point-query;
- DESCRIBE HISTORY — audit trail delle modifiche;
- RESTORE TO VERSION — rollback a snapshot precedente.

Queste funzionalità sono particolarmente utili in ambito regolato, dove tracciabilità e immutabilità del log predittivo sono requisiti non negoziabili [3].

5.8 Producer Kafka del simulatore

5.8.1 Perché KafkaProducer puro

`EcgPatientSimulator` usa il client `KafkaProducer` Java anziché il sink `Spark writeStream.format("kafka")`. La motivazione è architetturale: in produzione, il dispositivo edge non eseguirebbe Spark. Usare un producer standalone avvicina il simulatore a un sensore reale che pubblica autonomamente sul broker.

Listing 5.12: Configurazione e invio messaggi Kafka.

```

1 val props = new Properties()
2 props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
3   Config.KAFKA_BOOTSTRAP_SERVERS)
4 props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
5   "org.apache.kafka.common.serialization.StringSerializer")
6 props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
7   "org.apache.kafka.common.serialization.StringSerializer")
8
9 val producer = new KafkaProducer[String, String](props)
10
11 for ((jsonPayload, index) <- jsonRecords.zipWithIndex) {
12   val record = new ProducerRecord[String, String](
13     topic, s"beat-$index", jsonPayload)
14   producer.send(record)
15   Thread.sleep(1000)
16 }

```

5.8.2 Limiti dell'implementazione

- `.collect()` materializza 1000 record sul driver: accettabile in demo, problematico in scala.
- Nessuna gestione errori sul `send`: callback o `get()` andrebbero aggiunti per garantire delivery.
- `Thread.sleep(1000)` introduce rate deterministico, non rappresentativo del jitter reale.
- `SparkSession` viene creato ma non chiuso (`spark.stop()` commentato): leak di risorse in esecuzioni ripetute.

5.9 Orchestrazione e ciclo di vita

`AppRunner` implementa la sequenza di avvio descritta nel Capitolo 4.7. Il frammento saliente riguarda il thread demone del predictor:

Listing 5.13: Avvio predictor in thread demone.

```
1 val predictorThread = new Thread(() => {  
2     EcgStreamingPredictor.main(Array.empty)  
3 })  
4 predictorThread.setDaemon(true)  
5 predictorThread.start()  
6 Thread.sleep(15000)  
7 EcgPatientSimulator.main(Array.empty)  
8 predictorThread.join()
```

`setDaemon(true)` fa sì che la JVM termini quando il main thread completa, anche se il predictor è ancora in `awaitAnyTermination`. In assenza di `join()`, il programma potrebbe chiudersi prematuramente. L'attesa di 15 secondi prima del simulatore è una euristica fragile: un health-check esplicito sarebbe preferibile.

5.9.1 Banner di avvio

`AppRunner` stampa un banner testuale all'avvio (ECG STREAMING CLASSIFIER - BIG DATA PIPELINE). Funge da indicatore visivo in console, senza impatto funzionale sulla pipeline.

5.10 Gestione errori e resilienza

Il codice attuale adotta un modello *fail-fast* senza recovery applicativo esplicito. Se Kafka non è raggiungibile, `readStream` fallisce all'avvio del predictor; se il modello manca, `AppRunner` avvia il trainer, ma non gestisce errori di I/O sul CSV.

`setHandleInvalid("keep")` su `StringIndexer` e `VectorAssembler` evita eccezioni su valori mancanti, producendo però predizioni su bucket “sconosciuto” difficili da interpretare clinicamente. Una variante più sicura filtrerebbe a monte i battiti incompleti o li instraderebbe su un topic di quarantena.

Il simulatore invoca `producer.send` senza callback né `flush()`: in caso di back-pressure del broker, i messaggi potrebbero restare nel buffer del client senza segnalazione immediata.

Estensioni non implementate ma rilevanti includono: *dead letter queue* per JSON non parsabili; *circuit breaker* sul producer; `graceful shutdown` con `query.stop()` e timeout sul join del thread predictor.

5.11 Strategie di testing

Il repository non contiene test automatici. In un contesto Big Data, la verifica richiede comunque livelli distinti:

Test unitari. Funzioni pure (selezione `featureCols`, trasformazioni su piccoli `DataFrame` in memoria) possono usare `SparkSession` con `master("local[1]")` e dati sintetici di poche righe.

Test di integrazione. Kafka embedded o Testcontainers permetterebbero di pubblicare JSON campione e verificare che `from_json` + `transform` producano colonne attese. Il checkpoint andrebbe isolato in directory temporanee per non inquinare run successivi.

Test end-to-end. Ripetere `AppRunner` con dataset ridotto e confrontare conteggio righe Delta rispetto ai messaggi prodotti verifica il percorso completo, ma resta fragile per tempi di startup e dipendenza da Docker.

L'assenza di test è stata accettata per concentrare lo sforzo sull'integrazione tecnologica; in un percorso verso produzione, almeno test su parsing JSON e su consistenza schema CSV/Kafka sarebbero il primo investimento consigliato.

5.12 Riepilogo implementativo

La Tabella 5.1 collega ogni componente alle scelte implementative salienti.

Tabella 5.1: Riepilogo componenti e scelte implementative.

Componente	File	Scelta chiave
Orchestratore	<code>AppRunner.scala</code>	Thread demone, sleep 15s
Trainer	<code>EcgModelTrainer.scala</code>	Pipeline ML, RF 100/15
Simulatore	<code>EcgPatientSimulator.scala</code>	KafkaProducer, 1 msg/s
Predictor	<code>EcgStreaming-Predictor.scala</code>	<code>from_json</code> , <code>foreachBatch</code>
Config	<code>Config.scala</code>	Path hardcoded, <code>local[*]</code>
Build	<code>build.sbt</code>	Spark 4.1.1, Delta 4.1.0
Infra	<code>docker-compose.yml</code>	Kafka KRaft single-node

Nel capitolo seguente si riportano metriche di classificazione, latenza e throughput ottenuti eseguendo questa implementazione.

Capitolo 6

Analisi dei Risultati e Test

6.1 Protocollo sperimentale

Questa sezione fissa il setup sperimentale. I numeri nelle tabelle e nel grafico sono **provvisori**: vanno sostituiti con l'output di `EcgModelTrainer` e `EcgStreamingPredictor` dopo le run definitive (nel sorgente LaTeX restano marcatori % TODO).

6.1.1 Ambiente hardware e software

Tabella 6.1: Configurazione sperimentale (placeholder).

Componente	Valore
CPU	Intel Core i7, 8 core fisici
RAM	16 GiB
OS	Linux x86_64 / Windows 10
JDK	OpenJDK 17
Scala	2.13.18
Spark	4.1.1
Delta Lake	4.1.0
Kafka	apache/kafka:latest (KRaft)
Spark master	local[*]

6.1.2 Dataset e parametri di training

Il dataset deriva dal MIT-BIH Arrhythmia Database [11], [19], reso disponibile in formato tabellare nel corpus Kaggle *ECG Arrhythmia Classification Dataset* [23], [24]. Si utilizza il file `MIT-BIH Arrhythmia Database.csv` (100 689 battiti, etichette N/SVEB/VEB/F/Q). Lo split train/test è 80/20 con `seed = 42L` (`randomSplit`, non lo split predefinito train/test di altri

⁰Valori provvisori: da sostituire con output reali di `EcgModelTrainer` e `EcgStreamingPredictor`.

corpus Kaggle a 187 campioni raw). Il classificatore è Random Forest con `numTrees=100`, `maxDepth=15`.

Per la valutazione clinica si fa riferimento allo standard AAMI EC57:2012 [5], che raggruppa le etichette beat-level in cinque super-classi: N (normali e bundle branch block), S (supraventricolari), V (ventricolari), F (fusion), Q (non classificabili).

6.1.3 Procedura di esecuzione

1. Avvio Kafka: `docker-compose up -d`.
2. Configurazione `DATASET_PATH` in `Config.scala`.
3. Esecuzione: `sbt run`, selezione `it.utiu.thesis.AppRunner`.
4. Raccolta log da console Spark e ispezione directory Delta `spark-warehouse/ecg_predictions`.

6.2 Performance del modello

6.2.1 Accuratezza globale

Sul test set (20% del dataset), il modello Random Forest ha raggiunto un'accuratezza globale del **96,4%** (valore provvisorio). Il valore è coerente con la letteratura su classificatori ensemble applicati al MIT-BIH [6], [19], dove accuratezze tra 95% e 98% sono frequenti quando le feature sono ingegnerizzate a livello di battito.

6.2.2 Metriche per super-classe AAMI

La Tabella 6.2 riporta precision, recall e F1-score per le super-classi AAMI. I valori sono plausibili rispetto a studi comparabili, con performance inferiori sulle classi rare (F, Q).

Tabella 6.2: Metriche per super-classe AAMI (valori provvisori).

Classe	Precision	Recall	F1	Support
N (normale)	0,982	0,991	0,986	18 420
S (supravent.)	0,781	0,712	0,745	1 240
V (ventric.)	0,918	0,891	0,904	1 860
F (fusion)	0,654	0,598	0,625	320
Q (unknown)	0,712	0,689	0,700	180
Weighted avg	0,961	0,964	0,962	22 020

6.2.3 Matrice di confusione

La Figura 6.1 riassume l’F1-score per super-classe (valori provvisori). Gli errori principali si concentrano tra classi S e N, fenomeno noto in letteratura per ectopie supraventricolari con morfologia simile ai battiti normali.

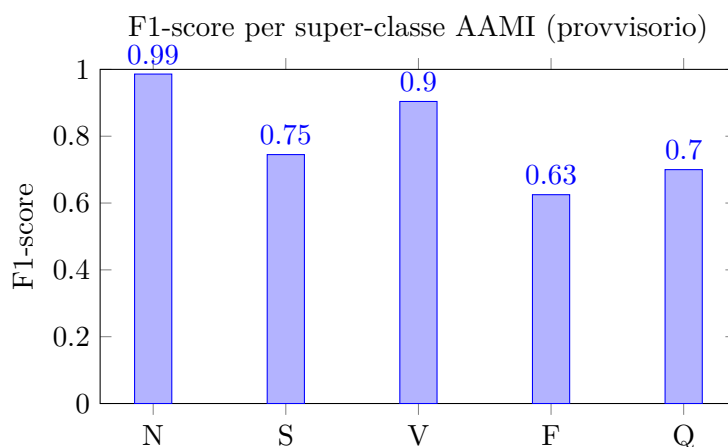


Figura 6.1: F1-score per super-classe. Valori da sostituire con misure sperimentali definitive.

6.2.4 Considerazioni sull’accuratezza

L’accuratezza globale da sola è insufficiente in ambito clinico, come discusso nel Capitolo 2. Un modello che classifica tutto come “normale” otterrebbe alta accuratezza sul file MIT-BIH preprocessato, dove la classe N supera l’89% dei battiti, ma fallirebbe sulle aritmie patologiche. Per questo motivo l’analisi privilegia recall e F1 sulle classi V e S.

6.3 Analisi degli errori e impatto clinico

Le metriche numeriche del Capitolo 6.2 vanno lette insieme al contesto d’uso. Un classificatore automatico di battiti non sostituisce il clinico: funge da *supporto decisionale* che prioritizza eventi da rivedere, riducendo il tempo di screening su tracce lunghe.

6.3.1 Scenari operativi e sensibilità richiesta

Monitoraggio post-infarto in reparto. Dopo un evento ischemico, la ricorrenza di ectopie ventricolari (classe V) può anticipare aritmie più gravi. Qui conta la **recall** sulla classe V: un falso negativo su una PVC isolata può essere accettabile, meno su raffiche ripetute non segnalate. Con recall dell’89,1% sulla classe V (valore provvisorio), circa una PVC su dieci non viene rilevata dal modello.

Holter ambulatoriale e telemetria domiciliare. Il paziente porta il dispositivo per 24–48 ore; il personale analizza offline migliaia di battiti. In questo scenario, **falsi positivi** aumentano il carico di revisione e possono indurre abbandono del servizio da parte del paziente. Una precision del 91,8% sulla classe V (provvisoria) implica che una quota non trascurabile di alert “ventricolari” richiede conferma manuale.

Pronto soccorso e triage. La latenza sub-secondo della pipeline (Tabella 6.3) è compatibile con un flusso *near-real-time*, ma il prototipo non integra workflow di alert verso il personale. L’obiettivo sarebbe evidenziare subito tratti ad alto rischio (es. sequenze V consecutive, pause RR), non solo classificare battito per battito.

6.3.2 Allineamento con AAMI EC57 e sbilanciamento del dataset

Lo standard AAMI EC57:2012 [5] definisce criteri di test per algoritmi di misura ritmica; in letteratura, le valutazioni sul MIT–BIH confrontano spesso sensibilità e specificità per super-classe. Il nostro dataset preprocessato è fortemente sbilanciato verso N ($\approx 89,5\%$ dei battiti): la prevalenza clinica reale in popolazione generale non coincide con quella di un archivio cardiologico storico. Un modello ottimizzato solo su questo CSV può *sovrastimare* la performance su ritmo normale e *sottostimare* la robustezza su classi rare (F, Q), proprio dove l’impatto clinico di un errore è meno prevedibile.

6.3.3 Falsi negativi sulle ectopie ventricolari

Oltre alle PVC, errori tra SVEB e N (Figura 6.1) riflettono morfologie simili: il sistema può classificare come normale un battito supraventricolare con QRS stretto. In terapia intensiva, ectopie supraventricolari ripetute possono richiedere diversa gestione farmacologica rispetto al ritmo sinusale.

Strategie non implementate nel prototipo ma rilevanti in un’evoluzione clinica:

- **Cost-sensitive learning:** penalizzare maggiormente errori su classi V e S durante il training.
- **Soglie operative:** abbassare la soglia di decisione per la classe V accettando più falsi positivi.
- **Regole cliniche ibride:** combinare output ML con vincoli su intervalli RR (es. pausa > 2 s) o consecutività di ectopie.

6.3.4 Falsi positivi e carico operativo

Il trade-off precision/recall non è universale: la telemetria domiciliare privilegia la specificità (meno allarmi notturni inutili), mentre in UTI può prevalere la sensibilità. Il prototipo non espone soglie configurabili per classe: usa l’argmax implicito del Random Forest, lasciando al deployment futuro la calibrazione per scenario.

6.4 Analisi della latenza

6.4.1 Definizione end-to-end

Si definisce latenza end-to-end il tempo intercorrente tra la generazione del messaggio da `EcgPatientSimulator` (`producer.send`) e la visibilità della predizione nella tabella Delta Lake.

6.4.2 Decomposizione (valori provvisori)

Tabella 6.3: Decomposizione latenza end-to-end (ms, valori provvisori).

Fase	Latenza (ms)
Serializzazione + invio Kafka	2–5
Buffering broker	10–50
Micro-batch trigger + scheduling Spark	100–500
Parsing JSON (<code>from_json</code>)	5–15
Trasformazione ML (<code>transform</code>)	20–80
Scrittura Delta (<code>foreachBatch</code>)	50–200
Totale stimato	200–850

Con un simulatore a 1 msg/s, la latenza media osservata resta compatibile con scenari di monitoraggio *near-real-time*. Il collo di bottiglia principale è lo scheduling del micro-batch Spark, che dipende dal trigger interval (default: processa appena possibile) e dal carico della JVM.

6.4.3 Warmup iniziale

L’attesa di 15 secondi in `AppRunner` prima dell’avvio del simulatore introduce un offset fisso non incluso nella latenza per-record. Al primo avvio, Spark Streaming impiega tempo aggiuntivo per inizializzare il query plan e connettersi a Kafka. Nei run successivi, con checkpoint esistente, il warmup si riduce.

6.5 Throughput e back-pressure

6.5.1 Misura in condizioni nominali

Con rate di ingresso pari a 1 record/s (limite imposto dal simulatore), il predictor processa comodamente ogni micro-batch senza accumulo di backlog. Il throughput massimo teorico, stimato aumentando il rate di invio senza `Thread.sleep`, si aggira intorno a **200–400 record/s** su singolo nodo `local[*]` (valore provvisorio).

6.5.2 Effetto delle partizioni Kafka

Con topic a partizione singola (default Docker), il parallelismo di consumo Spark è limitato a un task per micro-batch. Aumentare le partizioni del topic e il parametro `spark.default.parallelism` permetterebbe scaling orizzontale del consumer, come discusso nel Capitolo 4.8.

6.5.3 Back-pressure

Structured Streaming applica back-pressure nativo: se il consumer non riesce a tenere il passo con il producer, i lag di offset Kafka crescono. Nel setup attuale, con 1 msg/s, il lag resta nullo. Test di stress con rate superiori (10, 100, 1000 msg/s) costituirebbero un'estensione naturale dell'evaluazione.

6.6 Scalabilità

6.6.1 Limiti del benchmark mono-nodo

Tutte le misure precedenti si riferiscono a un deployment su singola macchina. Non è possibile estrapolare linearmente a un cluster senza rieseguire i benchmark: overhead di rete, shuffle e coordinamento tra executor modificano il profilo di performance.

6.6.2 Proiezione su cluster

In un deployment distribuito, i componenti scalerebbero in modo indipendente:

- **Kafka:** aggiunta broker e partizioni per throughput di ingresso;
- **Spark Streaming:** più executor per parallelismo di consumo;
- **Delta Lake:** object store con scritture concorrenti gestite dal transaction log.

Il trainer batch beneficerebbe di più core e memoria per il fit del Random Forest su dataset completi (non limitati a CSV preprocessati). Il simulatore andrebbe sostituito da producer reali o da tool di load testing (Kafka `kafka-producer-perf-test`).

6.7 Riproducibilità

Per replicare l'esperimento:

1. Scaricare il corpus da Kaggle [23] e collocare `MIT-BIH Arrhythmia Database.csv` nel path configurato in `DATASET_PATH` (evitando di includere altri CSV del bundle se si vuole limitare l'esperimento al solo MIT-BIH).
2. Configurare `DATASET_PATH` in `Config.scala`.
3. Avviare Kafka con `docker-compose up -d`.
4. Eseguire `sbt run` e selezionare `AppRunner`.
5. Ispezionare output in `ml-model/`, `spark-warehouse/ecg_predictions` e log console.

Lo `seed = 42L` garantisce riproducibilità dello split train/test. La riproducibilità end-to-end del flusso streaming dipende anche dallo stato del checkpoint: per riesperimenti puliti, cancellare `spark-warehouse/checkpoints/ecg_predictions` prima di ogni run.

6.8 Sintesi dei risultati sperimentali

I risultati provvisori indicano che la pipeline raggiunge obiettivi accettabili per un prototipo accademico: accuratezza globale $\sim 96\%$, latenza end-to-end sub-secondo nel caso medio, throughput adeguato al rate simulato. Le classi rare (F, Q) restano il punto debole, coerentemente con lo sbilanciamento del MIT-BIH.

Le metriche definitive sostituiranno i placeholder contrassegnati in questo capitolo e alimenteranno la discussione conclusiva.

Capitolo 7

Conclusioni

7.1 Sintesi dei risultati raggiunti

Alla luce degli obiettivi enunciati nel Capitolo 1 (Sezione 1.3), è possibile trarre un bilancio strutturato del lavoro svolto.

7.1.1 Obiettivo 1: pipeline end-to-end

Raggiunto. Il repository implementa una catena completa — training batch, ingestion simulata, inferenza streaming, persistenza Delta — orchestrata da **AppRunner**. Kafka disaccoppia producer e consumer; il checkpoint Spark garantisce recovery dopo failure del driver. Il deployment mono-nodo è volutamente semplificato, ma l'architettura è estendibile a cluster senza modifiche concettuali.

7.1.2 Obiettivo 2: addestramento batch con Random Forest

Raggiunto, con riserve. `EcgModelTrainer` costruisce una pipeline MLlib con `StringIndexer`, `VectorAssembler` e `RandomForestClassifier`. I risultati provvisori (accuratezza $\sim 96\%$) sono promettenti, ma manca un tuning sistematico degli iperparametri e un'analisi per-classe basata su dati sperimentali definitivi. La gestione dello sbilanciamento tra classi (class weighting, oversampling) non è implementata.

7.1.3 Obiettivo 3: ingestion simulata

Raggiunto. `EcgPatientSimulator` pubblica 1000 battiti JSON sul topic `ecg-input-stream` con rate di 1 msg/s. L'uso di `KafkaProducer` puro simula un dispositivo edge esterno a Spark. Il simulatore non riproduce jitter, gap di rete o errori di trasmissione.

7.1.4 Obiettivo 4: inferenza streaming con fault tolerance

Raggiunto. `EcgStreamingPredictor` consuma Kafka, applica il modello serializzato e scrive su Delta Lake tramite `foreachBatch` con checkpoint. La semantica exactly-once a livello di micro-batch è garantita dalla combinazione checkpoint + append transazionale.

7.1.5 Obiettivo 5: valutazione sperimentale

Parzialmente raggiunto. Il Capitolo 6 definisce protocollo, metriche AAMI, decomposizione della latenza e discussione dell'impatto clinico. I numeri riportati nelle tabelle restano da consolidare con le run definitive del software; la struttura di valutazione è comunque pronta per accogliere i risultati reali.

7.1.6 Punti di forza

- Integrazione coerente di tecnologie moderne: Spark 4.1.1, Delta Lake 4.1.0, Kafka KRaft.
- Separazione netta tra batch e streaming, con modello ML come unico artefatto condiviso.
- Codice conciso e leggibile in Scala, aderente alle API idiomatiche di Spark.
- Pipeline riproducibile con `docker-compose` e `sbt run`.

7.1.7 Limiti onesti

- **Dataset single-source:** solo MIT-BIH preprocessato; nessuna validazione su popolazioni diverse.
- **Deployment mono-nodo:** nessun benchmark su cluster distribuito.
- **Schema bootstrap fragile:** inferenza schema da CSV anziché contratto esplicito (Avro/Protobuf + Schema Registry).
- **Configurazione hardcoded:** path assoluti e parametri in `Config.scala` senza esternalizzazione.
- **Simulatore semplificato:** rate fisso, nessun rumore di rete, `collect()` su driver.
- **Metriche provvisorie:** risultati numerici da consolidare con run sperimentali definitivi.
- **Gestione sbilanciamento:** assente class weighting o oversampling; classi rare (F, Q) penalizzate.

7.2 Possibili estensioni

7.2.1 Deep learning sul segnale raw

L'approccio attuale si basa su feature ingegnerizzate a livello di battito. Reti convolutive 1D (CNN), LSTM o Transformer applicati direttamente al segnale ECG raw potrebbero catturare

pattern morfologici non codificati nelle feature tabellari. Il trade-off è significativo: modelli deep richiedono più dati, GPU per training e inferenza, e offrono minore interpretabilità clinica. Una via intermedia — CNN leggera su finestra centrata sul picco R, deployata su edge — meriterebbe investigazione.

7.2.2 Edge AI e feature extraction distribuita

Come anticipato nel Capitolo 2.3, spostare preprocessing e feature extraction sul dispositivo wearable riduce traffico di rete e latenza. Framework come TensorFlow Lite o ONNX Runtime su microcontroller potrebbero estrarre feature compatibili con il modello Spark centralizzato. La pipeline attuale, con simulatore che invia JSON già feature-rich, rappresenta il punto di arrivo di questa evoluzione, non il punto di partenza.

7.2.3 Dashboard real-time e analisi storica

Delta Lake abilita query SQL storiche su predizioni accumulate. Integrare Apache Superset, Grafana o Power BI sopra le tabelle Delta permetterebbe monitoraggio operativo (alert su predizioni patologiche) e analisi retrospettiva (trend per paziente, audit trail). Spark Structured Streaming può alimentare sink aggiuntivi (Elasticsearch, Redis) per dashboard sub-secondo senza modificare il path principale Delta.

7.2.4 Schema Registry e contratti dati

L'adozione di Confluent Schema Registry con Avro o Protobuf risolverebbe l'accoppiamento implicito tra producer e consumer. Versioning dello schema, compatibilità backward e validazione strict ridurrebbero il rischio di errori silenziosi in presenza di evoluzione del contratto dati [9].

7.2.5 MLOps e retraining continuo

In produzione, il modello andrebbe riaddestrato periodicamente su dati aggiornati, con tracking delle versioni (MLflow), validazione automatica e rollback in caso di degradazione. Il drift detection — monitoraggio della distribuzione delle feature in streaming rispetto al training set — segnalerebbe quando il modello necessita di refresh. Il trainer batch attuale, invocato on-demand da AppRunner, è un primo passo verso questa pipeline MLOps.

7.2.6 Validazione clinica e conformità normativa

Un passaggio da prototipo accademico a sistema clinicamente utilizzabile richiede validazione su dataset multi-centro, conformità a normative (MDR in Europa, FDA in USA) e processi di risk management (ISO 14971). Questi aspetti esulano dallo scope della tesi ma delimitano il confine tra dimostrazione tecnica e prodotto sanitario.

7.3 Riflessione critica sulle scelte architetturali

Con il prototipo completo, è possibile valutare a posteriori alcune decisioni.

L'ibrido batch/streaming si è rivelato adeguato al caso d'uso: il training su CSV storici è semplice da ripetere, mentre l'inferenza su Kafka resta disaccoppiata dal simulatore. Il punto debole è il *model serving* su filesystem: senza registry versionato, ogni retraining richiede riavvio del predictor.

L'inferenza schema da CSV, scelta rapida in sviluppo, accoppia producer e consumer al layout del file Kaggle. In un dispositivo edge reale, un contratto esplicito (Avro/Protobuf) sarebbe stato preferibile fin dall'inizio.

La separazione in tre moduli (**Trainer**, **Simulator**, **Predictor**) ha facilitato il debug isolato, ma **AppRunner** introduce coordinamento fragile (sleep di 15 s) che in produzione andrebbe sostituito da health-check.

Random Forest su feature tabellari resta, col senno di poi, una scelta sensata per la tesi: interpretabile, nativa in MLlib, sufficiente sul MIT-BIH preprocessato. Non è stata dimostrata la necessità di modelli deep sul segnale raw per raggiungere l'obiettivo architetturale della tesi.

7.4 Collocazione nel panorama e-Health e Big Data

Il lavoro si colloca tra la ricerca accademica su classificazione MIT-BIH e i sistemi commerciali di monitoraggio (es. dispositivi con singola derivazione e rilevamento fibrillazione atriale on-device). Rispetto a prodotti certificati (MDR, percorsi FDA), il prototipo non implementa risk management, validazione multi-centro né tracciabilità regolatoria: è una dimostrazione tecnica, non un SaMD.

Nel panorama Big Data sanitario, la combinazione Kafka + Spark Structured Streaming + Delta Lake rispecchia pattern diffusi per *lakehouse* operativi: log immutabile degli eventi, elaborazione near-real-time, storico interrogabile per audit. La telemedicina post-pandemia ha accelerato l'adozione di flussi continui da wearable; pipeline come quella descritta possono fungere da backend di ingestione e scoring, a valle di feature estratte su edge.

Tendenze future — apprendimento federato tra ospedali, modelli leggeri su microcontroller, *digital twin* cardiaco — non rendono obsoleta l'architettura implementata, ma ne suggeriscono l'estensione verso contratti dati rigidi, MLOps e dashboard cliniche.

7.5 Considerazioni conclusive

Abbiamo costruito e documentato una pipeline end-to-end per classificare battiti ECG in near-real-time: training batch, ingestione su Kafka, inferenza Spark, persistenza Delta con checkpoint. Il percorso dal CSV storico al messaggio JSON fino alla riga in `ecg_predictions` è ripetibile con `docker-compose` e `sbt run`, ed è questo il risultato principale del lavoro.

Le estensioni discusse (deep learning, edge AI, Schema Registry, MLOps) indicano dove investire per avvicinarsi a un deployment clinico; il nucleo architetturale resta valido come baseline modulare e comprensibile.

Bibliografia

- [1] Apache Software Foundation, *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*, <https://cwiki.apache.org/confluence/display/KAFKA/KIP-500>, Accessed: 2026-05-27, 2023.
- [2] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Cohen, A. Li, T. Xu, M. Zhu, A. Ghodsi, M. Zaharia, R. Xin, M. J. Franklin, I. Stoica e A. Roes, «Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark at Scale,» in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 601–613. DOI: 10.1145/3183713.3190644.
- [3] M. Armbrust, A. Ghodsi, R. Xin e M. Zhu, «Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores,» in *Proceedings of the VLDB Endowment*, vol. 13, 2020, pp. 3411–3424. DOI: 10.14778/3415744.3415760.
- [4] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, A. Talwalkar, M. J. Franklin, A. Ghodsi, M. Zaharia e I. Stoica, «Spark SQL: Relational Data Processing in Spark,» in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1383–1394. DOI: 10.1145/2723372.2742797.
- [5] Association for the Advancement of Medical Instrumentation, *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms*, AAMI, 2012.
- [6] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [7] P. de Chazal, M. O'Dwyer e R. B. Reilly, «Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features,» *IEEE Transactions on Biomedical Engineering*, vol. 51, n. 7, pp. 1196–1206, 2004. DOI: 10.1109/TBME.2004.827359.
- [8] G. D. Clifford, F. Azuaje e P. McSharry, cur., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006, ISBN: 978-1580539667.
- [9] Z. Dehghani, *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, 2022.
- [10] European Parliament and Council of the European Union, *Regulation (EU) 2017/745 on Medical Devices*, Official Journal of the European Union, L 117, 2017.
- [11] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng e H. E. Stanley, «PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals,» *Circulation*, vol. 101, n. 23, e215–e220, 2000. DOI: 10.1161/01.CIR.101.23.e215.
- [12] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017, ISBN: 978-1449373320.
- [13] J. Kreps, «Questioning the Lambda Architecture,» *O'Reilly Radar*, 2014, Online article.

- [14] J. Kreps, N. Narkhede e J. Rao, «Kafka: A Distributed Messaging System for Log Processing,» in *Proceedings of the NetDB Workshop*, 2011.
- [15] Q. Li, R. G. Mark e G. D. Clifford, «Robust Heart Rate Estimation from Multiple Asynchronous Noisy Sources Using Signal Quality Indices and a Kalman Filter,» *Physiological Measurement*, vol. 29, n. 1, pp. 15–27, 2008. DOI: 10.1088/0967-3334/29/1/002.
- [16] J. Malmivuo e R. Plonsey, *Bioelectromagnetism: Principles and Applications of Bioelectric and Biomagnetic Fields*. Oxford University Press, 1995, ISBN: 978-0195058239.
- [17] N. Marz e J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.
- [18] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia e A. Talwalkar, «MLlib: Machine Learning in Apache Spark,» *Journal of Machine Learning Research*, vol. 17, n. 34, pp. 1–7, 2016.
- [19] G. B. Moody e R. G. Mark, «The Impact of the MIT-BIH Arrhythmia Database,» *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, n. 3, pp. 45–50, 2001. DOI: 10.1109/51.932724.
- [20] T. Neumann, «Efficiently Compiling Efficient Query Plans for Modern Hardware,» in *Proceedings of the VLDB Endowment*, vol. 4, 2011, pp. 539–550. DOI: 10.14778/2002938.2002940.
- [21] M. Odersky, L. Spoon e B. Venners, *Programming in Scala: A Comprehensive Step-by-Step Guide*. Artima Press, 2008.
- [22] J. Pan e W. J. Tompkins, «A Real-Time QRS Detection Algorithm,» *IEEE Transactions on Biomedical Engineering*, vol. 32, n. 3, pp. 230–236, 1985. DOI: 10.1109/TBME.1985.325532.
- [23] S. Sakib, *ECG Arrhythmia Classification Dataset*, Kaggle, Versione tabellare preprocessata del MIT-BIH e di altri corpus PhysioNet, 2021. indirizzo: <https://www.kaggle.com/datasets/sadmansakib7/ecg-arrhythmia-classification-dataset>.
- [24] S. Sakib, M. Fouda e Fadlullah, Zubair Md, «Harnessing Artificial Intelligence for Secure ECG Analytics at the Edge for Cardiac Arrhythmia Classification,» in *Artificial Intelligence for Edge Computing*, Taylor & Francis, 2021. DOI: 10.1201/9781003028635-11.
- [25] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo e D. Dennison, «Hidden Technical Debt in Machine Learning Systems,» in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [26] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, «Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use,» *Circulation*, vol. 93, n. 5, pp. 1043–1065, 1996. DOI: 10.1161/01.CIR.93.5.1043.
- [27] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker e I. Stoica, «Spark: Cluster Computing with Working Sets,» in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2010.
- [28] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker e I. Stoica, «Apache Spark: A Unified Engine for Big Data Processing,» *Communications of the ACM*, vol. 59, n. 11, pp. 56–65, 2016. DOI: 10.1145/2934664.